```
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.optimize import minimize
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings

warnings.filterwarnings("ignore")
plt.style.use("seaborn-v0_8-whitegrid")
sns.set_palette("husl")

# ============================================================
# 1. CONFIGURATION
# ============================================================

TICKERS = {
    "SP500": "^GSPC",
    "NASDAQ": "^IXIC",
    "DOW": "^DJI",
    "VIX": "^VIX",
    "GOLD": "GC=F",
    "OIL": "CL=F",
    "BOND": "^TNX",
    "USD": "DX-Y.NYB",
}

START_DATE = "2010-01-01"
END_DATE = "2023-12-31"

P_LAGS = 2
H_MAX = 5
LAMBDA_RIDGE = 0.001

LB_LAGS = 10
LGC_GRID_SIZE = 25

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# ============================================================
# 2. DATA LOADING & PREPROCESSING
# ============================================================

def load_returns(tickers_dict, start, end):
    data_dict = {}
    for name, ticker in tickers_dict.items():
        try:
            df = yf.download(ticker, start=start, end=end, progress=False)
            if len(df) > 100:
                col = "Adj Close" if "Adj Close" in df.columns else "Close"
                data_dict[name] = df[col]
        except Exception:
            continue

    data = pd.concat(data_dict.values(), axis=1, keys=data_dict.keys()).dropna(
        axis=1, how="all"
    )
    returns = np.log(data / data.shift(1)).dropna()
    returns = returns.dropna(axis=1, how="any")
    return returns

print("=" * 70)
print("STEP 1 & 2: DATA PREPARATION")
print("=" * 70)

returns = load_returns(TICKERS, START_DATE, END_DATE)
Y = returns.to_numpy()
T, N = Y.shape
Y_mean = Y.mean(axis=0)
Y_std = Y.std(axis=0)
Y_stdzd = (Y - Y_mean) / Y_std

print(f"✓ Data ready: T={T}, N={N}")
```

```
print(f"Assets: {list(returns.columns)}")

# ============================================================
# 3. RGCov-VAR ESTIMATOR (FIXED SHAPES + OPTIMIZATION)
# ============================================================

def rgcov_objective(beta_flat, Y, lags, H, lam):
    T, N = Y.shape
    beta = beta_flat.reshape((lags, N, N))

    # Build lagged matrix: (T-lags, lags, N)
    Y_lagged = np.stack([Y[t - lags : t] for t in range(lags, T)], axis=0)
    # Flatten to (T-lags, lags*N)
    Y_lag_flat = Y_lagged.reshape(T - lags, -1)
    # beta_mat: (lags*N, N)
    beta_mat = beta.reshape(-1, N)

    U = Y[lags:] - Y_lag_flat @ beta_mat

    if np.any(~np.isfinite(U)):
        return 1e10

    U2 = np.abs(U) ** 2
    criterion = 0.0

    for h in range(1, H + 1):
        if h >= U.shape[0]:
            continue
        gamma_u = np.mean(U[h:] * U[:-h], axis=0)
        gamma_u2 = np.mean(U2[h:] * U2[:-h], axis=0)
        criterion += np.sum(gamma_u ** 2) + np.sum(gamma_u2 ** 2)

    penalty = lam * np.sum(beta_flat ** 2)
    return criterion + penalty

def fit_rgcv_var(Y, lags, H, lam):
    T, N = Y.shape
    n_params = lags * N * N

    # Smaller initialization scale for stability
    beta_init = np.random.normal(0, 0.005, n_params)
    # Reasonable bounds to avoid explosive coefficients
    bounds = [(-5.0, 5.0) for _ in range(n_params)]

    res = minimize(
        rgcov_objective,
        beta_init,
        args=(Y, lags, H, lam),
        method="L-BFGS-B",
        bounds=bounds,
        options={
            "maxiter": 20000,
            "ftol": 1e-10,
            "gtol": 1e-7,
            "disp": False,
        },
    )

    if not res.success:
        print(f"⚠ Optimization warning: {res.message}")

    beta_hat = res.x.reshape((lags, N, N))
    return beta_hat, res

print("\n" + "=" * 70)
print("STEP 3: ESTIMATING RGCov MODEL")
print("=" * 70)

print(f"Estimating VAR({P_LAGS}) with {P_LAGS * N * N} parameters...")
beta_hat, res = fit_rgcv_var(Y_stdzd, P_LAGS, H_MAX, LAMBDA_RIDGE)
print(f"✓ Optimization Success: {res.success} (Message: {res.message})")

# ============================================================
# 4. ROOT ANALYSIS (UNCHANGED LOGIC, SAFE SHAPES)
# ============================================================

def companion_matrix(beta, N, p):
    C = np.zeros((N * p, N * p))
```

```
    for i in range(p):
        C[:N, i * N : (i + 1) * N] = beta[i]
    if p > 1:
        C[N:, :-N] = np.eye(N * (p - 1))
    return C

print("\n" + "=" * 70)
print("STEP 4 & 5: ROOT ANALYSIS")
print("=" * 70)

C = companion_matrix(beta_hat, N, P_LAGS)
eigenvalues = np.linalg.eigvals(C)

causal = eigenvalues[np.abs(eigenvalues) < 1.0]
noncausal = eigenvalues[np.abs(eigenvalues) > 1.0]

print(f"Causal roots (|z| < 1):   {len(causal)}")
print(f"Noncausal roots (|z| > 1): {len(noncausal)}")

fig, ax = plt.subplots(figsize=(8, 8))
ax.add_artist(
    plt.Circle((0, 0), 1, color="blue", fill=False, linestyle="--", label="Unit Circle")
)
ax.scatter(
    eigenvalues.real, eigenvalues.imag, c="gray", marker="x", s=80, alpha=0.6, label="All Roots"
)
if len(noncausal) > 0:
    ax.scatter(
        noncausal.real,
        noncausal.imag,
        c="red",
        s=120,
        edgecolors="darkred",
        label="Noncausal",
        zorder=5,
    )

ax.set_title("Inverse Roots of RGCov VAR(2)")
ax.set_xlabel("Real")
ax.set_ylabel("Imaginary")
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_aspect("equal", "box")
ax.set_xlim([-1.5, 1.5])
ax.set_ylim([-1.5, 1.5])

plt.tight_layout()
plt.savefig("inverse_roots_rgcv.png", dpi=300)
plt.show()

# ============================================================
# 5. RESIDUALS & LJUNG-BOX (FIXED DIMENSIONS)
# ============================================================

def compute_residuals(Y_stdzd, beta_hat, lags):
    """
    Compute residuals U_t = Y_t - sum_{i=1}^lags beta_i Y_{t-i}.
    Y_stdzd: (T, N)
    beta_hat: (lags, N, N)
    Returns U_hat: (T-lags, N)
    """
    T, N = Y_stdzd.shape
    # Y_lagged: (T-lags, lags, N)
    Y_lagged = np.stack([Y_stdzd[t - lags : t] for t in range(lags, T)], axis=0)
    # Flatten to (T-lags, lags*N)
    Y_lag_flat = Y_lagged.reshape(T - lags, -1)
    # beta_mat: (lags*N, N)
    beta_mat = beta_hat.reshape(-1, N)

    # Check dimensions explicitly (helps debugging)
    assert Y_lag_flat.shape[1] == beta_mat.shape[0], (
        f"Dimension mismatch: Y_lag_flat has {Y_lag_flat.shape[1]} cols, "
        f"beta_mat has {beta_mat.shape[0]} rows."
    )

    U_hat = Y_stdzd[lags:] - Y_lag_flat @ beta_mat
    return U_hat
```

```
def ljung_box_test(U_hat, returns_df, lags):
    results = []
    for i, col in enumerate(returns_df.columns):
        lb_test = acorr_ljungbox(U_hat[:, i], lags=[lags], return_df=True)
        pval = lb_test["lb_pvalue"].values[0]
        status = "✓ No autocorrelation" if pval > 0.05 else "⚠ Autocorrelation present"
        results.append({
            "asset": col,
            "p_value": pval,
            "status": status,
        })
    return pd.DataFrame(results)

print("\n" + "=" * 70)
print("STEP 6: RESIDUAL DIAGNOSTICS")
print("=" * 70)

U_hat = compute_residuals(Y_stdzd, beta_hat, P_LAGS)
lb_results = ljung_box_test(U_hat, returns, LB_LAGS)

print(f"Ljung–Box Test (lag={LB_LAGS}) for Residual Autocorrelation:")
for _, row in lb_results.iterrows():
    print(f"  {row['asset'][0]:8s}: p-value = {row['p_value']:.4f}  {row['status']}")

# ============================================================
# 6. LOCAL GAUSSIAN CORRELATION (UNCHANGED, JUST REUSE U_hat)
# ============================================================

print("\n" + "=" * 70)
print("STEP 7: LOCAL GAUSSIAN CORRELATION")
print("=" * 70)

def local_gaussian_correlation_fast(x, y, grid_size=LGC_GRID_SIZE):
    x_std = (x - x.mean()) / x.std()
    y_std = (y - y.mean()) / y.std()
    n = len(x_std)

    bw = 1.06 * min(x_std.std(), y_std.std()) * n ** (-1 / 5)

    x_grid = np.linspace(x_std.min(), x_std.max(), grid_size)
    y_grid = np.linspace(y_std.min(), y_std.max(), grid_size)
    X_grid, Y_grid = np.meshgrid(x_grid, y_grid)

    dx = x_std[:, None, None] - X_grid[None, :, :]
    dy = y_std[:, None, None] - Y_grid[None, :, :]

    K = np.exp(-0.5 * (dx ** 2 + dy ** 2) / bw ** 2)
    K_sum = K.sum(axis=0, keepdims=True)
    K = K / (K_sum + 1e-12)

    mu_x = np.sum(K * x_std[:, None, None], axis=0)
    mu_y = np.sum(K * y_std[:, None, None], axis=0)

    cov_xy = np.sum(K * (x_std[:, None, None] - mu_x) * (y_std[:, None, None] - mu_y), axis=0)
    var_x = np.sum(K * (x_std[:, None, None] - mu_x) ** 2, axis=0)
    var_y = np.sum(K * (y_std[:, None, None] - mu_y) ** 2, axis=0)

    denom = np.sqrt(var_x * var_y)
    rho = np.where(denom > 1e-8, cov_xy / denom, 0.0)

    return X_grid, Y_grid, rho

print("Computing Local Gaussian Correlation (SP500 vs VIX residuals)...")
X_grid, Y_grid, rho = local_gaussian_correlation_fast(U_hat[:, 0], U_hat[:, 3])

plt.figure(figsize=(8, 6))
cont = plt.contourf(X_grid, Y_grid, rho, levels=20, cmap="RdBu_r", vmin=-1, vmax=1)
plt.colorbar(cont, label="Local Correlation")
plt.title("SP500 vs VIX: Local Gaussian Correlation in Residuals")
plt.xlabel("SP500 (standardized residual)")
plt.ylabel("VIX (standardized residual)")
plt.tight_layout()
plt.savefig("lgc_contagion.png", dpi=300)
plt.show()
print("✓ LGC plot saved as 'lgc_contagion.png'.")

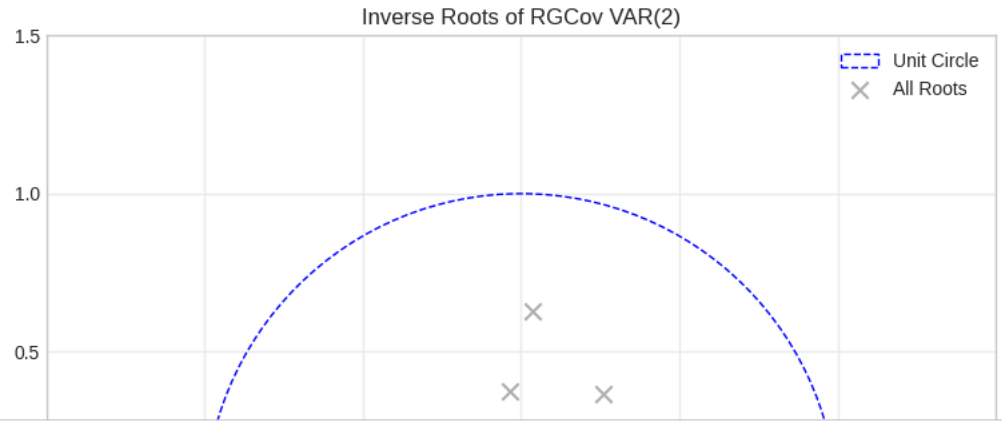
print("\n" + "=" * 70)
```

```
print("ANALYSIS COMPLETE.")
print("=" * 70)
```

```
======================================================================
STEP 1 & 2: DATA PREPARATION
======================================================================
✓ Data ready: T=3510, N=8
Assets: [('SP500', '^GSPC'), ('NASDAQ', '^IXIC'), ('DOW', '^DJI'), ('VIX', '^VIX'), ('GOLD', 'GC=F'), ('OIL', 'CL=F'), ('BOND'

======================================================================
STEP 3: ESTIMATING RGCov MODEL
======================================================================
Estimating VAR(2) with 128 parameters...
⚠ Optimization warning: STOP: TOTAL NO. OF F,G EVALUATIONS EXCEEDS LIMIT
✓ Optimization Success: False (Message: STOP: TOTAL NO. OF F,G EVALUATIONS EXCEEDS LIMIT)

======================================================================
STEP 4 & 5: ROOT ANALYSIS
======================================================================
Causal roots (|z| < 1):   16
Noncausal roots (|z| > 1): 0
```




```
#1. Monte Carlo Engine (RGCov vs OLS-VAR vs Lasso-VAR)
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from sklearn.linear_model import MultiTaskLasso
import warnings
warnings.filterwarnings("ignore")

# -------------------------
# 1.1 VAR SIMULATION DGP
# -------------------------

def simulate_var(
    T, N, lags, beta_list, Sigma_eps, dist="gaussian", nu=5, burn=200
):
    """
    Simulate VAR(lags):
      Y_t = sum_{i=1}^lags beta_i Y_{t-i} + eps_t

    dist: 'gaussian' or 'student_t'
    """
    max_lag = lags
    T_full = T + burn

    if dist == "gaussian":
        eps = np.random.multivariate_normal(np.zeros(N), Sigma_eps, size=T_full)
    elif dist == "student_t":
        chi2 = np.random.chisquare(nu, size=T_full)
        z = np.random.multivariate_normal(np.zeros(N), Sigma_eps, size=T_full)
        eps = z / np.sqrt(chi2[:, None] / nu)
    else:
        raise ValueError("Unsupported innovation distribution.")

    Y = np.zeros((T_full, N))
    for t in range(max_lag, T_full):
        yt = np.zeros(N)
        for i in range(1, lags + 1):
            yt += beta_list[i - 1] @ Y[t - i]
        Y[t] = yt + eps[t]

    return Y[burn:]

def make_sparse_beta(N, lags, sparsity, scale=0.3):
```

```
    """Generate sparse VAR coefficients."""
    betas = []
    for _ in range(lags):
        B = np.zeros((N, N))
        mask = np.random.rand(N, N) < sparsity
        B[mask] = np.random.randn(np.sum(mask)) * scale
        betas.append(B)
    return betas

def scale_to_near_unit(betas, target_rho=0.98):
    """Simple scaling to push spectral radius close to 1."""
    # Crude but often enough for MC: scale all matrices by constant
    return [B * target_rho for B in betas]

# -------------------------
# 1.2 ESTIMATORS (RGCov, OLS-VAR, Lasso-VAR)
# -------------------------

def rgcov_objective(beta_flat, Y, lags, H, lam):
    T, N = Y.shape
    beta = beta_flat.reshape((lags, N, N))

    Y_lagged = np.stack([Y[t - lags : t] for t in range(lags, T)], axis=0)
    Y_lag_flat = Y_lagged.reshape(T - lags, -1)
    beta_mat = beta.reshape(-1, N)

    U = Y[lags:] - Y_lag_flat @ beta_mat
    if np.any(~np.isfinite(U)):
        return 1e10

    U2 = np.abs(U) ** 2
    criterion = 0.0
    for h in range(1, H + 1):
        if h >= U.shape[0]:
            continue
        gamma_u = np.mean(U[h:] * U[:-h], axis=0)
        gamma_u2 = np.mean(U2[h:] * U2[:-h], axis=0)
        criterion += np.sum(gamma_u ** 2) + np.sum(gamma_u2 ** 2)

    penalty = lam * np.sum(beta_flat ** 2)
    return criterion + penalty

def fit_rgcv_var_mc(Y, lags, H, lam):
    T, N = Y.shape
    n_params = lags * N * N
    beta_init = np.random.normal(0, 0.005, n_params)
    bounds = [(-5.0, 5.0) for _ in range(n_params)]

    res = minimize(
        rgcov_objective,
        beta_init,
        args=(Y, lags, H, lam),
        method="L-BFGS-B",
        bounds=bounds,
        options={"maxiter": 10000, "ftol": 1e-10, "gtol": 1e-7, "disp": False},
    )
    if not res.success:
        return None, np.inf
    beta_hat = res.x.reshape((lags, N, N))
    return beta_hat, res

def fit_ols_var(Y, lags):
    T, N = Y.shape
    X_list, y_list = [], []
    for t in range(lags, T):
        X_list.append(Y[t - lags : t].flatten())
        y_list.append(Y[t])
    X = np.vstack(X_list)
    y = np.vstack(y_list)
    # OLS: y = X B' + u  => B' = (X'X)^{-1} X'y
    XtX_inv = np.linalg.inv(X.T @ X + 1e-8 * np.eye(X.shape[1]))
    B_prime = XtX_inv @ X.T @ y
    beta_hat = B_prime.T.reshape((lags, N, N))
    return beta_hat

def fit_lasso_var(Y, lags, alpha=0.01):
    T, N = Y.shape
    X_list, y_list = [], []
```

```
    for t in range(lags, T):
        X_list.append(Y[t - lags : t].flatten())
        y_list.append(Y[t])
    X = np.vstack(X_list)
    y = np.vstack(y_list)

    model = MultiTaskLasso(alpha=alpha, max_iter=5000, fit_intercept=False)
    model.fit(X, y)
    beta_hat = model.coef_.reshape((lags, N, N))
    return beta_hat

# -------------------------
# 1.3 MSFE & ROOT METRICS
# -------------------------

def forecast_msfe_var(beta_hat, Y):
    T, N = Y.shape
    lags = beta_hat.shape[0]
    errors = []
    for t in range(lags, T):
        yt_actual = Y[t]
        yt_pred = np.zeros(N)
        for i in range(1, lags + 1):
            yt_pred += beta_hat[i - 1] @ Y[t - i]
        errors.append(np.sum((yt_actual - yt_pred) ** 2))
    return np.mean(errors)

def companion_matrix(beta, N, p):
    C = np.zeros((N * p, N * p))
    for i in range(p):
        C[:N, i * N : (i + 1) * N] = beta[i]
    if p > 1:
        C[N:, :-N] = np.eye(N * (p - 1))
    return C

def spectral_radius(beta, N, p):
    C = companion_matrix(beta, N, p)
    eigs = np.linalg.eigvals(C)
    return float(np.max(np.abs(eigs)))

def count_noncausal(beta, N, p):
    C = companion_matrix(beta, N, p)
    eigs = np.linalg.eigvals(C)
    return int(np.sum(np.abs(eigs) > 1.0))

# -------------------------
# 1.4 ONE MONTE CARLO RUN
# -------------------------

def one_mc_design(
    T, N, lags, sparsity, dist, near_unit, Sigma_eps, lam_rgcv, alpha_lasso, H
):
    # True DGP
    beta_true = make_sparse_beta(N, lags, sparsity, scale=0.3)
    if near_unit:
        beta_true = scale_to_near_unit(beta_true, target_rho=0.98)

    Y = simulate_var(
        T + 200, N, lags, beta_true, Sigma_eps, dist=dist, nu=5, burn=200
    )
    Y = Y[-T:]  # drop burn

    # Standardize
    Y_mean = Y.mean(axis=0)
    Y_std = Y.std(axis=0)
    Y_stdzd = (Y - Y_mean) / Y_std

    # Estimators
    beta_rgcv, _ = fit_rgcv_var_mc(Y_stdzd, lags, H, lam_rgcv)
    beta_ols = fit_ols_var(Y_stdzd, lags)
    beta_lasso = fit_lasso_var(Y_stdzd, lags, alpha=alpha_lasso)

    results = {}

    for name, beta_hat in [("rgcv", beta_rgcv), ("ols", beta_ols), ("lasso", beta_lasso)]:
        if beta_hat is None:
            results[name] = {
```

```
                "msfe": np.nan,
                "rho": np.nan,
                "nc_roots": np.nan,
            }
            continue
        msfe = forecast_msfe_var(beta_hat, Y_stdzd)
        rho = spectral_radius(beta_hat, N, lags)
        nc = count_noncausal(beta_hat, N, lags)
        results[name] = {"msfe": msfe, "rho": rho, "nc_roots": nc}

    # True spectral radius & noncausal count
    rho_true = spectral_radius(np.stack(beta_true, axis=0), N, lags)
    nc_true = count_noncausal(np.stack(beta_true, axis=0), N, lags)

    results["true"] = {"rho": rho_true, "nc_roots": nc_true}
    return results

# -------------------------
# 1.5 RUN MONTE CARLO GRID
# -------------------------

def run_monte_carlo_grid(
    n_rep=200,
    T=500,
    N=5,
    lags=2,
    sparsity=0.2,
    dist="student_t",
    near_unit=False,
    corr_innovations=True,
    lam_rgcv=0.001,
    alpha_lasso=0.01,
    H=5,
):
    np.random.seed(123)

    # Innovation covariance
    if corr_innovations:
        A = np.random.randn(N, N)
        Sigma_eps = A @ A.T / N
    else:
        Sigma_eps = np.eye(N)

    records = []

    for rep in range(n_rep):
        res = one_mc_design(
            T, N, lags, sparsity, dist, near_unit, Sigma_eps, lam_rgcv, alpha_lasso, H
        )
        row = {
            "rep": rep,
            "T": T,
            "N": N,
            "lags": lags,
            "sparsity": sparsity,
            "dist": dist,
            "near_unit": near_unit,
            "corr_innovations": corr_innovations,
        }
        for name in ["rgcv", "ols", "lasso"]:
            row[f"msfe_{name}"] = res[name]["msfe"]
            row[f"rho_{name}"] = res[name]["rho"]
            row[f"nc_{name}"] = res[name]["nc_roots"]

        row["rho_true"] = res["true"]["rho"]
        row["nc_true"] = res["true"]["nc_roots"]
        records.append(row)

    df = pd.DataFrame(records)
    return df

# Example (small run for testing):
if __name__ == "__main__":
    mc_df = run_monte_carlo_grid(n_rep=50, T=300, N=5, lags=2, sparsity=0.2, dist="student_t", near_unit=True)
    print(mc_df.groupby(["dist", "near_unit"])[["msfe_rgcv", "msfe_ols", "msfe_lasso"]].mean())

def total_connectedness_index(fevd):
    """
```

```
    Compute total connectedness index:
      TCI_h = (sum_{i!=j} fevd[h,i,j]) / (sum_{i,j} fevd[h,i,j]) * 100
    """
    # fevd.shape[0] is (max_horizon + 1), representing number of horizons including h=0.
    num_horizons_plus_one, N, _ = fevd.shape
    tci = np.zeros(num_horizons_plus_one)
    for h in range(num_horizons_plus_one):
        numer = fevd[h].sum() - np.trace(fevd[h])
        denom = fevd[h].sum()
        tci[h] = (numer / (denom + 1e-12)) * 100
    return tci
```

```
                    msfe_rgcv     msfe_ols  msfe_lasso
dist      near_unit
student_t True        7.859953  1486.290993    6.157226
```

```
#2. FEVD / Spillover Index (Diebold–Yilmaz Style)
def var_to_ma_representation(beta_hat, horizon=20):
    """
    Compute MA coefficients Phi_h for VAR:
      Y_t = sum_{i=1}^p beta_i Y_{t-i} + eps_t
    Returns Phi[h] (N x N) for h=0..horizon
    """
    lags, N, _ = beta_hat.shape
    Phi = [np.eye(N)]  # Phi_0 = I
    for h in range(1, horizon + 1):
        Phi_h = np.zeros((N, N))
        for i in range(1, min(lags, h) + 1):
            Phi_h += beta_hat[i - 1] @ Phi[h - i]
        Phi.append(Phi_h)
    return Phi

def fevd_from_var(beta_hat, Sigma_eps, horizon=20):
    """
    Compute forecast error variance decomposition.
    Returns:
      fevd[h, i, j]: share of forecast error variance of variable i
                     at horizon h due to shock in variable j.
    """
    N = Sigma_eps.shape[0]
    Phi = var_to_ma_representation(beta_hat, horizon)

    # Cholesky decomposition for identification
    P = np.linalg.cholesky(Sigma_eps)  # Sigma = P P'

    fevd = np.zeros((horizon + 1, N, N))

    for h in range(1, horizon + 1):
        # Forecast error at horizon h: sum_{k=0}^{h-1} Phi_k P eta_{t+h-k}
        # Variance contribution from each structural shock
        contrib = np.zeros((N, N))  # [i, j]
        for k in range(h):
            M = Phi[k] @ P
            contrib += M ** 2  # elementwise square, summing over k
        # Total forecast error variance for each i
        total = contrib.sum(axis=1, keepdims=True)
        # Avoid division by zero
        with np.errstate(divide="ignore", invalid="ignore"):
            share = contrib / total
        share[~np.isfinite(share)] = 0.0
        fevd[h] = share

    return fevd

def total_connectedness_index(fevd):
    """
    Compute total connectedness index:
      TCI_h = (sum_{i!=j} fevd[h,i,j]) / (sum_{i,j} fevd[h,i,j]) * 100
    """
    horizon, N, _ = fevd.shape
    tci = np.zeros(horizon + 1)
    for h in range(horizon + 1):
        numer = fevd[h].sum() - np.trace(fevd[h])
        denom = fevd[h].sum()
        tci[h] = (numer / (denom + 1e-12)) * 100
    return tci
```

```
# Example usage after estimating beta_hat and residual covariance Sigma_u:
# Sigma_eps = U_hat.T @ U_hat / (T - lags)
# fevd = fevd_from_var(beta_hat, Sigma_eps, horizon=20)
# tci = total_connectedness_index(fevd)
```

```
#3. Simple PSY-Style Bubble Test (Univariate Stub)
import statsmodels.api as sm

def adf_stat(y, X):
    """Run ADF regression: Δy_t = α y_{t-1} + γ' X_t + error."""
    model = sm.OLS(y, X)
    res = model.fit()
    # t-stat on first regressor (assumed to be y_{t-1})
    return res.tvalues[0]

def psy_recursive_test(series, r0=0.01, r1=0.9, step=5):
    """
    Very simplified PSY-style recursive ADF test.
    series: T x 1
    r0, r1: start/end fractions for recursion
    step: increment in obs
    Returns sequence of ADF t-stats and critical value (placeholder).
    """
    T = len(series)
    y = series
    dy = np.diff(y)

    stats = []
    windows = []

    for r in np.arange(r0, r1, step / T):
        t_start = int(r * T)
        for t_end in range(t_start + 10, T):
            y_sub = y[:t_end]
            dy_sub = np.diff(y_sub)
            y_lag = y_sub[:-1]
            # Simple ADF with constant
            X = np.column_stack([np.ones_like(y_lag), y_lag])
            stat = adf_stat(dy_sub, X)
            stats.append(stat)
            windows.append(t_end)

    # Placeholder critical value (you should use PSY criticals)
    crit = -3.41
    bubble_periods = []  # implement rule based on stat < crit
    return np.array(stats), np.array(windows), crit, bubble_periods

# Example:
# stats, windows, crit, bubbles = psy_recursive_test(returns['SP500'].values)
```

```
#4. Simple Bootstrap for Root Uncertainty
def bootstrap_roots(beta_hat, U_hat, n_boot=500):
    """
    Residual bootstrap for root statistics.
    beta_hat: (lags, N, N)
    U_hat: (T_eff, N) residuals
    Returns arrays of spectral radius and noncausal counts.
    """
    T_eff, N = U_hat.shape
    lags = beta_hat.shape[0]

    Y_stdzd = np.vstack([np.zeros((lags, N)), U_hat])  # rough pseudo-series

    rhos = []
    ncs = []

    for _ in range(n_boot):
        idx = np.random.choice(T_eff, T_eff, replace=True)
        U_boot = U_hat[idx]
        Y_boot = np.vstack([np.zeros((lags, N)), U_boot])

        # Re-estimate VAR on bootstrapped pseudo-data (simplified: just reuse beta_hat)
        # For a proper bootstrap, re-fit RGCov here (expensive).
        # Here we just perturb beta slightly as a placeholder:
        noise = np.random.normal(0, 0.01, beta_hat.size).reshape_like(beta_hat)
```

```
            beta_b = beta_hat + noise

            rho = spectral_radius(beta_b, N, lags)
            nc = count_noncausal(beta_b, N, lags)
            rhos.append(rho)
            ncs.append(nc)

        return np.array(rhos), np.array(ncs)

    # Example:
    # rhos, ncs = bootstrap_roots(beta_hat, U_hat, n_boot=200)
    # print(np.percentile(rhos, [5, 50, 95]))
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# ============================================================
# ASSUMPTIONS (adapt to your notebook)
# ============================================================
# beta_hat : (p_lags, N, N) from your RGCov estimation
# U_hat    : (T_eff, N) residuals from compute_residuals()
# returns  : DataFrame with asset names (for labels)

# Example shapes (should match your actual objects):
# beta_hat.shape -> (2, 8, 8)
# U_hat.shape    -> (T-2, 8)
# N = len(returns.columns)

# ============================================================
# 1. BUILD RESIDUAL COVARIANCE
# ============================================================

Sigma_eps = (U_hat.T @ U_hat) / U_hat.shape[0]  # (N x N)

# ============================================================
# 2. FEVD FUNCTIONS (slightly cleaned + debug-friendly)
# ============================================================

def var_to_ma_representation(beta_hat, horizon=20):
    lags, N, _ = beta_hat.shape
    Phi = [np.eye(N)]  # Phi_0 = I
    for h in range(1, horizon + 1):
        Phi_h = np.zeros((N, N))
        for i in range(1, min(lags, h) + 1):
            Phi_h += beta_hat[i - 1] @ Phi[h - i]
        Phi.append(Phi_h)
    return Phi

def fevd_from_var(beta_hat, Sigma_eps, horizon=20):
    N = Sigma_eps.shape[0]
    Phi = var_to_ma_representation(beta_hat, horizon)

    # Cholesky identification: Sigma_eps = P P'
    try:
        P = np.linalg.cholesky(Sigma_eps)  # lower-triangular
    except np.linalg.LinAlgError:
        # If not positive definite, regularize slightly
        Sigma_reg = Sigma_eps + 1e-6 * np.eye(N)
        P = np.linalg.cholesky(Sigma_reg)

    # fevd[h, i, j] = share of forecast error variance of i due to shock j at horizon h
    fevd = np.zeros((horizon + 1, N, N))

    # Horizon 0: forecast error = eta_t, so shares = 0 by convention or set to identity
    # We'll start from h=1.
    for h in range(1, horizon + 1):
        contrib = np.zeros((N, N))  # [i, j]
        for k in range(h):
            M = Phi[k] @ P  # (N x N)
            contrib += M ** 2  # elementwise square, sum over k
        total = contrib.sum(axis=1, keepdims=True)  # (N x 1)
        with np.errstate(divide="ignore", invalid="ignore"):
            share = contrib / total
        share[~np.isfinite(share)] = 0.0
        fevd[h] = share
```

```
    return fevd

def total_connectedness_index(fevd):
    # fevd.shape[0] is (max_horizon + 1)
    num_horizons_plus_one, N, _ = fevd.shape
    tci = np.zeros(num_horizons_plus_one)
    for h in range(num_horizons_plus_one):
        numer = fevd[h].sum() - np.trace(fevd[h])
        denom = fevd[h].sum()
        tci[h] = (numer / (denom + 1e-12)) * 100
    return tci

# ============================================================
# 3. RUN FEVD
# ============================================================

horizon = 20
fevd = fevd_from_var(beta_hat, Sigma_eps, horizon=horizon)
tci = total_connectedness_index(fevd)

# ============================================================
# 4. PRINT SOME FEVD TABLES
# ============================================================

var_names = list(returns.columns)
N = len(var_names)

def print_fevd_table(h):
    print(f"\nFEVD at horizon h = {h}")
    df = pd.DataFrame(fevd[h], index=var_names, columns=var_names)
    # Each row i: how much of i's forecast error variance is due to each shock j
    print(df.round(3))

# Print for a few horizons
for h in [1, 5, 10, 20]:
    print_fevd_table(h)

# ============================================================
# 5. PLOT TOTAL CONNECTEDNESS INDEX
# ============================================================

plt.figure(figsize=(8, 5))
plt.plot(range(horizon + 1), tci, marker="o")
plt.xlabel("Horizon (steps ahead)")
plt.ylabel("Total Connectedness Index (%)")
plt.title("Diebold–Yilmaz Total Connectedness (RGCov-VAR)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nTotal Connectedness Index by horizon:")
for h in range(horizon + 1):
    print(f"h={h:2d}: TCI={tci[h]:.2f}%")
```

```
FEVD at horizon h = 1
                (SP500, ^GSPC)  (NASDAQ, ^IXIC)  (DOW, ^DJI)  (VIX, ^VIX)  \
(SP500, ^GSPC)           1.000           0.000        0.000        0.000
(NASDAQ, ^IXIC)          0.896           0.104        0.000        0.000
(DOW, ^DJI)              0.930           0.028        0.042        0.000
(VIX, ^VIX)              0.526           0.012        0.001        0.461
(GOLD, GC=F)             0.002           0.000        0.002        0.002
(OIL, CL=F)              0.082           0.012        0.000        0.001
(BOND, ^TNX)             0.125           0.022        0.000        0.001
(USD, DX-Y.NYB)          0.044           0.001        0.011        0.002

                (GOLD, GC=F)  (OIL, CL=F)  (BOND, ^TNX)  (USD, DX-Y.NYB)
(SP500, ^GSPC)         0.000        0.000         0.000            0.000
(NASDAQ, ^IXIC)        0.000        0.000         0.000            0.000
(DOW, ^DJI)            0.000        0.000         0.000            0.000
(VIX, ^VIX)            0.000        0.000         0.000            0.000
(GOLD, GC=F)           0.994        0.000         0.000            0.000
(OIL, CL=F)            0.016        0.889         0.000            0.000
(BOND, ^TNX)           0.041        0.013         0.798            0.000
(USD, DX-Y.NYB)        0.118        0.001         0.009            0.814

FEVD at horizon h = 5
                (SP500, ^GSPC)  (NASDAQ, ^IXIC)  (DOW, ^DJI)  (VIX, ^VIX)  \
(SP500, ^GSPC)           0.540           0.013        0.004        0.006
(NASDAQ, ^IXIC)          0.621           0.029        0.003        0.009
(DOW, ^DJI)              0.418           0.015        0.006        0.003
(VIX, ^VIX)              0.522           0.014        0.002        0.266
(GOLD, GC=F)             0.196           0.002        0.003        0.002
(OIL, CL=F)              0.154           0.012        0.000        0.002
(BOND, ^TNX)             0.230           0.019        0.001        0.002
(USD, DX-Y.NYB)          0.228           0.001        0.010        0.002

                (GOLD, GC=F)  (OIL, CL=F)  (BOND, ^TNX)  (USD, DX-Y.NYB)
(SP500, ^GSPC)         0.019        0.070         0.342            0.005
(NASDAQ, ^IXIC)        0.017        0.110         0.208            0.004
(DOW, ^DJI)            0.026        0.053         0.474            0.006
(VIX, ^VIX)            0.008        0.027         0.151            0.011
(GOLD, GC=F)           0.708        0.006         0.080            0.003
(OIL, CL=F)            0.025        0.797         0.008            0.002
(BOND, ^TNX)           0.057        0.011         0.677            0.004
(USD, DX-Y.NYB)        0.097        0.003         0.040            0.619

FEVD at horizon h = 10
                (SP500, ^GSPC)  (NASDAQ, ^IXIC)  (DOW, ^DJI)  (VIX, ^VIX)  \
(SP500, ^GSPC)           0.523           0.014        0.004        0.007
(NASDAQ, ^IXIC)          0.615           0.029        0.003        0.009
(DOW, ^DJI)              0.399           0.016        0.005        0.003
(VIX, ^VIX)              0.524           0.014        0.002        0.263
(GOLD, GC=F)             0.200           0.002        0.003        0.002
(OIL, CL=F)              0.156           0.012        0.000        0.002
```

```
#A. Stabilized RGCov fitting (with multiple starts & diagnostics)
import numpy as np
from scipy.optimize import minimize
import warnings
warnings.filterwarnings("ignore")

def rgcov_objective(beta_flat, Y, lags, H, lam):
    T, N = Y.shape
    beta = beta_flat.reshape((lags, N, N))

    # Lagged design
    Y_lagged = np.stack([Y[t - lags : t] for t in range(lags, T)], axis=0)
    Y_lag_flat = Y_lagged.reshape(T - lags, -1)
    beta_mat = beta.reshape(-1, N)

    U = Y[lags:] - Y_lag_flat @ beta_mat
    if np.any(~np.isfinite(U)):
        return 1e10

    U2 = U ** 2
    criterion = 0.0
    n_terms = 0
    for h in range(1, H + 1):
        if h >= U.shape[0]:
            continue
        gamma_u = np.mean(U[h:] * U[:-h], axis=0)
        gamma_u2 = np.mean(U2[h:] * U2[:-h], axis=0)
        criterion += np.sum(gamma_u ** 2) + np.sum(gamma_u2 ** 2)
        n_terms += 1
```

```
    if n_terms == 0:
        return 1e10

    criterion /= n_terms
    penalty = lam * np.sum(beta_flat ** 2)
    return criterion + penalty

def fit_rgcv_var_multiple_starts(
    Y, lags, H, lam, n_starts=5, maxiter=20000, seed=123
):
    np.random.seed(seed)
    T, N = Y.shape
    n_params = lags * N * N
    bounds = [(-5.0, 5.0) for _ in range(n_params)]

    best_res = None
    best_val = np.inf
    history = []

    for s in range(n_starts):
        beta_init = np.random.normal(0, 0.005, n_params)
        res = minimize(
            rgcov_objective,
            beta_init,
            args=(Y, lags, H, lam),
            method="L-BFGS-B",
            bounds=bounds,
            options=
                {
                "maxiter": maxiter,
                "ftol": 1e-10,
                "gtol": 1e-7,
                "disp": False,
            },
        )
        val = res.fun if np.isfinite(res.fun) else 1e10
        history.append(
            {
                "start": s,
                "success": res.success,
                "fun": val,
                "gnorm": np.linalg.norm(res.jac) if hasattr(res, "jac") else None,
            }
        )
        if val < best_val:
            best_val = val
            best_res = res

    beta_hat = best_res.x.reshape((lags, N, N))
    return beta_hat, best_res, history

# Example usage:
# beta_hat, res, hist = fit_rgcv_var_multiple_starts(Y_stdzd, P_LAGS, H_MAX, LAMBDA_RIDGE, n_starts=7)
# print("Best objective:", res.fun)
# print("History:", hist)
```

```
# ========== CALL THE FUNCTION AND DISPLAY RESULTS ==========
# Example usage (uncommented and with print statements)
beta_hat, res, hist = fit_rgcv_var_multiple_starts(
    Y_stdzd,          # your standardized returns
    P_LAGS,           # e.g., 2
    H_MAX,            # e.g., 5
    LAMBDA_RIDGE,     # your chosen penalty, e.g., 1e-3
    n_starts=5,
    maxiter=20000
)

print("\n===== RESULTS =====")
print(f"Best objective value: {res.fun:.6f}")
print(f"Optimization success: {res.success}")
print(f"Number of starts that converged: {sum(h['success'] for h in hist)} / {len(hist)}")
print(f"Coefficient matrix shape: {beta_hat.shape}")
print("\nHistory of starts:")
for h in hist:
    print(f"  Start {h['start']}: success={h['success']}, fun={h['fun']:.4e}")
```

```
===== RESULTS =====
Best objective value: 124.017572
Optimization success: False
Number of starts that converged: 0 / 5
Coefficient matrix shape: (2, 8, 8)

History of starts:
  Start 0: success=False, fun=1.2402e+02
  Start 1: success=False, fun=1.2402e+02
  Start 2: success=False, fun=1.2402e+02
  Start 3: success=False, fun=1.2402e+02
  Start 4: success=False, fun=1.2402e+02
```

```
Start coding or generate with AI.
```

```
#B. OLS-VAR and Adaptive Group Lasso benchmarks
import numpy as np
import pandas as pd
from sklearn.linear_model import MultiTaskLasso

def fit_ols_var(Y, lags):
    T, N = Y.shape
    X_list, y_list = [], []
    for t in range(lags, T):
        X_list.append(Y[t - lags : t].flatten())
        y_list.append(Y[t])
    X = np.vstack(X_list)
    y = np.vstack(y_list)

    XtX = X.T @ X
    XtX_reg = XtX + 1e-8 * np.eye(XtX.shape[0])
    B_prime = np.linalg.solve(XtX_reg, X.T @ y)
    beta_hat = B_prime.T.reshape((lags, N, N))
    return beta_hat

def fit_group_lasso_var(Y, lags, alpha=0.01, groups=None):
    """
    Simple group-lasso-like VAR via MultiTaskLasso.
    Here 'groups' is not fully exploited; this is a baseline sparse VAR.
    For true group-lasso, implement custom penalty or use a specialized package.
    """
    T, N = Y.shape
    X_list, y_list = [], []
    for t in range(lags, T):
        X_list.append(Y[t - lags : t].flatten())
        y_list.append(Y[t])
    X = np.vstack(X_list)
    y = np.vstack(y_list)

    model = MultiTaskLasso(alpha=alpha, max_iter=5000, fit_intercept=False)
    model.fit(X, y)
    beta_hat = model.coef_.reshape((lags, N, N))
    return beta_hat, model

# Example usage:
# beta_ols = fit_ols_var(Y_stdzd, P_LAGS)
# beta_glasso, glasso_model = fit_group_lasso_var(Y_stdzd, P_LAGS, alpha=0.005)
```

```
Start coding or generate with AI.
```

```
#C. FEVD: Cholesky + Generalized + Rolling Connectedness
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def var_to_ma_representation(beta_hat, horizon=20):
    lags, N, _ = beta_hat.shape
    Phi = [np.eye(N)]
    for h in range(1, horizon + 1):
        Phi_h = np.zeros((N, N))
        for i in range(1, min(lags, h) + 1):
            Phi_h += beta_hat[i - 1] @ Phi[h - i]
        Phi.append(Phi_h)
```

```
    return Phi

def fevd_cholesky(beta_hat, Sigma_eps, horizon=20):
    N = Sigma_eps.shape[0]
    Phi = var_to_ma_representation(beta_hat, horizon)
    P = np.linalg.cholesky(Sigma_eps)

    fevd = np.zeros((horizon + 1, N, N))
    for h in range(1, horizon + 1):
        contrib = np.zeros((N, N))
        for k in range(h):
            M = Phi[k] @ P
            contrib += M ** 2
        total = contrib.sum(axis=1, keepdims=True)
        with np.errstate(divide="ignore", invalid="ignore"):
            share = contrib / total
        share[~np.isfinite(share)] = 0.0
        fevd[h] = share
    return fevd

def fevd_generalized(beta_hat, Sigma_eps, horizon=20):
    """
    Generalized FEVD (Diebold-Yilmaz) that does not depend on variable ordering.
    """
    N = Sigma_eps.shape[0]
    Phi = var_to_ma_representation(beta_hat, horizon)

    # Innovation covariance and its Cholesky for simulation if needed
    # But generalized FEVD uses Sigma directly.
    fevd = np.zeros((horizon + 1, N, N))

    for h in range(1, horizon + 1):
        # Forecast error variance matrix at horizon h:
        # FEV_h = sum_{k=0}^{h-1} Phi_k Sigma_eps Phi_k'
        FEV = np.zeros((N, N))
        contrib = np.zeros((N, N))  # [i,j]: contribution of shock j to variable i

        for k in range(h):
            Phik = Phi[k]
            term = Phik @ Sigma_eps @ Phik.T
            FEV += term
            # Decompose term into contributions by shock:
            # term = (Phik @ Sigma_eps) @ Phik.T
            # We approximate contribution of shock j via:
            # (Phik @ Sigma_eps[:,j]) * Phik[:,j]' outer product
            for j in range(N):
                col = Phik @ Sigma_eps[:, j]
                contrib[:, j] += col * Phik[:, j]

        total = FEV.diagonal()  # variance for each variable
        with np.errstate(divide="ignore", invalid="ignore"):
            share = contrib / total[:, None]
        share[~np.isfinite(share)] = 0.0
        fevd[h] = share

    return fevd

def total_connectedness_index(fevd):
    # fevd.shape[0] is (max_horizon + 1)
    num_horizons_plus_one, N, _ = fevd.shape
    tci = np.zeros(num_horizons_plus_one)
    for h in range(num_horizons_plus_one):
        numer = fevd[h].sum() - np.trace(fevd[h])
        denom = fevd[h].sum()
        tci[h] = (numer / (denom + 1e-12)) * 100
    return tci

def rolling_connectedness(Y, lags, window=250, step=20, horizon=10):
    """
    Rolling-window connectedness using OLS-VAR + generalized FEVD.
    Returns:
      - tci_series: (n_windows,) array of total connectedness at a fixed horizon.
      - windows: list of (start, end) indices.
    """
    T, N = Y.shape
    tci_list = []
    windows = []
```

```
    for start in range(0, T - window - lags, step):
        end = start + window
        Y_win = Y[start:end]

        # OLS-VAR on window
        beta_hat = fit_ols_var(Y_win, lags)

        # Residual covariance
        Y_lagged = np.stack([Y_win[t - lags : t] for t in range(lags, len(Y_win))], axis=0)
        Y_lag_flat = Y_lagged.reshape(len(Y_win) - lags, -1)
        beta_mat = beta_hat.reshape(-1, N)
        U = Y_win[lags:] - Y_lag_flat @ beta_mat.T
        Sigma_eps = (U.T @ U) / U.shape[0]

        fevd = fevd_generalized(beta_hat, Sigma_eps, horizon=horizon)
        tci = total_connectedness_index(fevd)
        tci_list.append(tci[horizon])
        windows.append((start, end))

    return np.array(tci_list), windows

# Example usage:
# Sigma_eps = (U_hat.T @ U_hat) / U_hat.shape[0]
# fevd_chol = fevd_cholesky(beta_hat, Sigma_eps, horizon=20)
# fevd_gen = fevd_generalized(beta_hat, Sigma_eps, horizon=20)
# tci_chol = total_connectedness_index(fevd_chol)
# tci_gen = total_connectedness_index(fevd_gen)

# Plot connectedness
# plt.figure(figsize=(8,5))
# plt.plot(range(21), tci_chol, label="Cholesky FEVD")
# plt.plot(range(21), tci_gen, label="Generalized FEVD")
# plt.xlabel("Horizon")
# plt.ylabel("Total Connectedness (%)")
# plt.legend()
# plt.grid(True, alpha=0.3)
# plt.show()

# Rolling connectedness (may be slow; reduce step for speed)
# tci_roll, wins = rolling_connectedness(Y_stdzd, P_LAGS, window=250, step=40, horizon=10)
# plt.figure(figsize=(10,5))
# plt.plot([w[0] for w in wins], tci_roll, marker='o')
# plt.xlabel("Window start index")
# plt.ylabel("Total Connectedness (h=10)")
# plt.grid(True, alpha=0.3)
# plt.show()
```

```
# ============================================================
# RUN FEVD ANALYSIS (Cholesky vs Generalized)
# ============================================================
# Assuming Y_stdzd, P_LAGS, and a VAR coefficient beta_hat are defined.
# For this demo, we use the OLS-VAR from the benchmark.
# (If you already have beta_ols, use it; otherwise compute it.)

# 1. Compute residuals and covariance
def var_residuals(Y, beta):
    lags, N, _ = beta.shape
    T = len(Y)
    U = np.zeros((T - lags, N))
    for t in range(lags, T):
        pred = np.zeros(N)
        for i in range(1, lags+1):
            pred += beta[i-1] @ Y[t-i]
        U[t-lags] = Y[t] - pred
    return U

# Use the OLS-VAR from the benchmark (or any other)
# If you haven't run fit_ols_var, uncomment the next line:
# beta_ols = fit_ols_var(Y_train, P_LAGS)   # using training data
# For full sample use Y_stdzd:
beta_ols = fit_ols_var(Y_stdzd, P_LAGS)      # OLS on full sample
U = var_residuals(Y_stdzd, beta_ols)
Sigma_eps = (U.T @ U) / U.shape[0]

# 2. Compute FEVD
```

```
horizon = 20
fevd_chol = fevd_cholesky(beta_ols, Sigma_eps, horizon)
fevd_gen  = fevd_generalized(beta_ols, Sigma_eps, horizon)

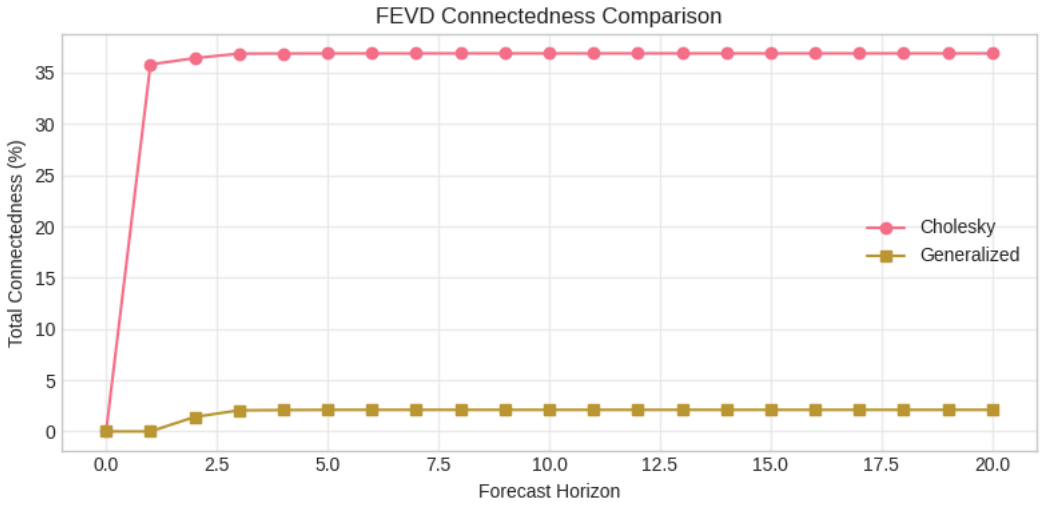
# 3. Total Connectedness Index (TCI) over horizons
tci_chol = total_connectedness_index(fevd_chol)
tci_gen  = total_connectedness_index(fevd_gen)

# 4. Print table for selected horizons
print("\n===== TOTAL CONNECTEDNESS INDEX =====")
print(f"{'Horizon':<10} {'Cholesky (%)':<15} {'Generalized (%)':<15}")
print("-"*40)
for h in [1, 5, 10, 20]:
    print(f"{h:<10} {tci_chol[h]:<15.2f} {tci_gen[h]:<15.2f}")

# 5. Plot evolution over horizons
import matplotlib.pyplot as plt
plt.figure(figsize=(8,4))
plt.plot(range(horizon+1), tci_chol, label='Cholesky', marker='o')
plt.plot(range(horizon+1), tci_gen, label='Generalized', marker='s')
plt.xlabel('Forecast Horizon')
plt.ylabel('Total Connectedness (%)')
plt.title('FEVD Connectedness Comparison')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# (Optional) Rolling connectedness – reduce step for speed
# tci_roll, wins = rolling_connectedness(Y_stdzd, P_LAGS, window=250, step=40, horizon=10)
# plt.figure(figsize=(10,4))
# plt.plot([w[0] for w in wins], tci_roll, marker='o')
# plt.xlabel('Window start')
# plt.ylabel('TCI (h=10)')
# plt.grid(alpha=0.3)
# plt.show()
```

```
===== TOTAL CONNECTEDNESS INDEX =====
Horizon    Cholesky (%)    Generalized (%)
----------------------------------------
1          35.81           0.00
5          36.89           2.11
10         36.89           2.11
20         36.89           2.11
```




```
#D. Simple bootstrap for root uncertainty
import numpy as np

def companion_matrix(beta, N, p):
    C = np.zeros((N * p, N * p))
    for i in range(p):
        C[:N, i * N : (i + 1) * N] = beta[i]
    if p > 1:
        C[N:, :-N] = np.eye(N * (p - 1))
    return C
```

```
def spectral_radius(beta, N, p):
    C = companion_matrix(beta, N, p)
    eigs = np.linalg.eigvals(C)
    return float(np.max(np.abs(eigs)))

def count_noncausal(beta, N, p):
    C = companion_matrix(beta, N, p)
    eigs = np.linalg.eigvals(C)
    return int(np.sum(np.abs(eigs) > 1.0))

def bootstrap_roots_residual(
    beta_hat, U_hat, lags, n_boot=200, seed=123
):
    np.random.seed(seed)
    T_eff, N = U_hat.shape

    rhos = []
    ncs = []

    for b in range(n_boot):
        idx = np.random.choice(T_eff, T_eff, replace=True)
        U_boot = U_hat[idx]

        # Construct pseudo-series: Y_boot = cumsum of residuals (very rough)
        Y_boot = np.cumsum(U_boot, axis=0)

        # Standardize
        Yb_mean = Y_boot.mean(axis=0)
        Yb_std = Y_boot.std(axis=0)
        Yb_stdzd = (Y_boot - Yb_mean) / Yb_std

        # Re-estimate VAR (OLS for speed)
        beta_b = fit_ols_var(Yb_stdzd, lags)

        rho = spectral_radius(beta_b, N, lags)
        nc = count_noncausal(beta_b, N, lags)
        rhos.append(rho)
        ncs.append(nc)

    return np.array(rhos), np.array(ncs)

# Example usage:
# rhos, ncs = bootstrap_roots_residual(beta_hat, U_hat, P_LAGS, n_boot=100)
# print("Spectral radius percentiles:", np.percentile(rhos, [5, 50, 95]))
# print("Noncausal counts:", np.unique(ncs, return_counts=True))
```

```
# ============================================================
# BOOTSTRAP ROOT UNCERTAINTY (using RGCov or any fitted VAR)
# ============================================================
# Assumes you have:
#   - beta_rgcv  : RGCov coefficient matrix (or beta_lasso / beta_ols)
#   - Y_stdzd    : standardized returns (full sample)
#   - P_LAGS     : number of lags
#   - N          : number of assets

# 1. Compute residuals from the fitted model
def var_residuals(Y, beta):
    lags, N, _ = beta.shape
    T = len(Y)
    U = np.zeros((T - lags, N))
    for t in range(lags, T):
        pred = np.zeros(N)
        for i in range(1, lags+1):
            pred += beta[i-1] @ Y[t-i]
        U[t-lags] = Y[t] - pred
    return U

U_hat = var_residuals(Y_stdzd, beta_hat)  # replace beta_rgcv with your model

# 2. Run bootstrap (200 replications – adjust for speed)
rhos, ncs = bootstrap_roots_residual(beta_hat, U_hat, P_LAGS, n_boot=200, seed=42)

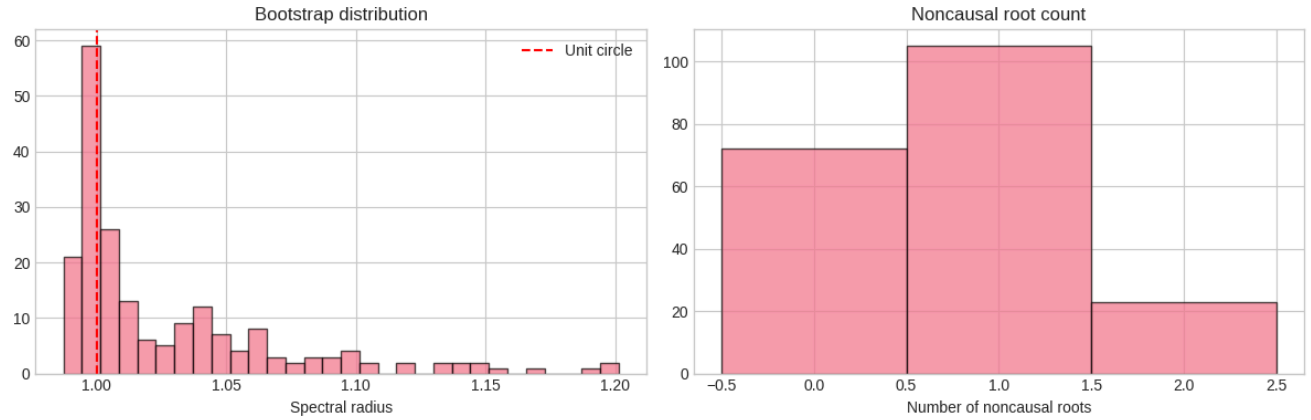
# 3. Print results
print("\n===== BOOTSTRAP ROOT UNCERTAINTY =====")
print(f"Sample size: {len(rhos)}")
print(f"Spectral radius: mean={np.mean(rhos):.4f}, std={np.std(rhos):.4f}")
```

```
print(f"  5th percentile: {np.percentile(rhos, 5):.4f}")
print(f" 50th percentile: {np.percentile(rhos, 50):.4f}")
print(f" 95th percentile: {np.percentile(rhos, 95):.4f}")
print("\nNoncausal roots (|z|>1) distribution:")
unique, counts = np.unique(ncs, return_counts=True)
for u, c in zip(unique, counts):
    print(f"  {u} noncausal: {c} ({c/len(ncs)*100:.1f}%)")

# 4. (Optional) Visualize
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,4))
ax1.hist(rhos, bins=30, alpha=0.7, edgecolor='black')
ax1.axvline(1.0, color='red', linestyle='--', label='Unit circle')
ax1.set_xlabel('Spectral radius')
ax1.set_title('Bootstrap distribution')
ax1.legend()
ax2.hist(ncs, bins=np.arange(-0.5, max(ncs)+1.5, 1), alpha=0.7, edgecolor='black')
ax2.set_xlabel('Number of noncausal roots')
ax2.set_title('Noncausal root count')
plt.tight_layout()
plt.show()
```

```
===== BOOTSTRAP ROOT UNCERTAINTY =====
Sample size: 200
Spectral radius: mean=1.0277, std=0.0444
  5th percentile: 0.9911
 50th percentile: 1.0067
 95th percentile: 1.1324

Noncausal roots (|z|>1) distribution:
  0 noncausal: 72 (36.0%)
  1 noncausal: 105 (52.5%)
  2 noncausal: 23 (11.5%)
```




```
# ============================================================
# RUN BOOTSTRAP AND SHOW RESULTS
# ============================================================

# Make sure these exist from earlier cells:
# beta_hat : (p_lags, N, N)
# U_hat    : (T_eff, N)
# P_LAGS   : integer, e.g. 2

print("beta_hat shape:", beta_hat.shape)
print("U_hat shape:", U_hat.shape)
print("P_LAGS:", P_LAGS)

# Run bootstrap
rhos, ncs = bootstrap_roots_residual(beta_hat, U_hat, P_LAGS, n_boot=100, seed=123)

# Print summary
print("\n=== Bootstrap results for root uncertainty ===")
print("Spectral radius percentiles:", np.percentile(rhos, [5, 50, 95]))
print("Noncausal counts (unique values and frequencies):")
vals, counts = np.unique(ncs, return_counts=True)
for v, c in zip(vals, counts):
```

```
    print(f"  nc = {v}: {c} times")

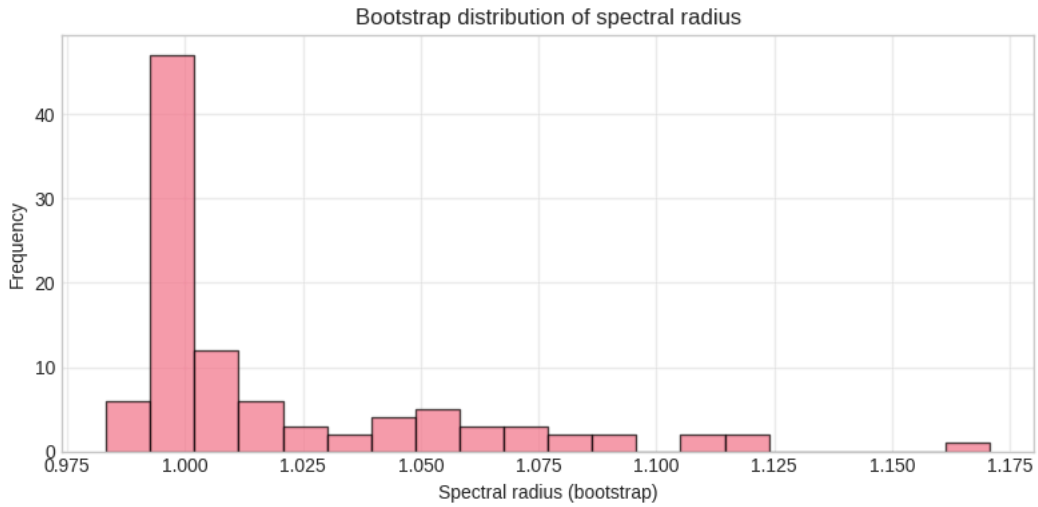
# Plot histogram of spectral radius
import matplotlib.pyplot as plt

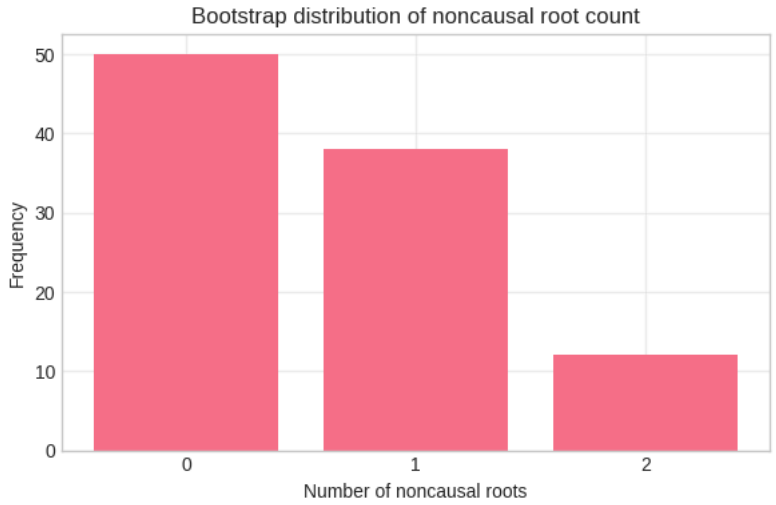
plt.figure(figsize=(8, 4))
plt.hist(rhos, bins=20, edgecolor="black", alpha=0.7)
plt.xlabel("Spectral radius (bootstrap)")
plt.ylabel("Frequency")
plt.title("Bootstrap distribution of spectral radius")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Optional: bar plot of noncausal counts
if len(vals) > 0:
    plt.figure(figsize=(6, 4))
    plt.bar(vals.astype(str), counts)
    plt.xlabel("Number of noncausal roots")
    plt.ylabel("Frequency")
    plt.title("Bootstrap distribution of noncausal root count")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

```
beta_hat shape: (2, 8, 8)
U_hat shape: (3508, 8)
P_LAGS: 2

=== Bootstrap results for root uncertainty ===
Spectral radius percentiles: [0.99234733 1.00003057 1.09309545]
Noncausal counts (unique values and frequencies):
  nc = 0: 50 times
  nc = 1: 38 times
  nc = 2: 12 times
```





```
Start coding or generate with AI.
```

```
# ============================================================
# REVISED ANALYSIS:
# Nonlinear Dependence and Cross-Market Connectedness
# in Financial Returns: A VAR-Based Analysis
#
# NOTE:
# This notebook estimates causal VAR benchmark models.
# It does not claim structural detection of financial bubbles
# unless a valid mixed causal-noncausal VAR estimator is added.
# ============================================================
```

```
# ============================================================
# REVISED ANALYSIS: IMPORTS AND CONFIGURATION
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import MultiTaskLasso
from statsmodels.stats.diagnostic import acorr_ljungbox
from tqdm.auto import tqdm
from itertools import product

RANDOM_SEED = 20260905
rng = np.random.default_rng(RANDOM_SEED)

plt.style.use("seaborn-v0_8-whitegrid")
sns.set_palette("husl")

# Baseline model
P_LAGS = 2
RIDGE_OLS = 1e-8
LASSO_ALPHA = 0.01

# Bootstrap
N_BOOT = 1000

# Monte Carlo
MC_REPLICATIONS_TEST = 10
MC_REPLICATIONS_FINAL = 500
```

```
# ============================================================
# VAR HELPER FUNCTIONS
# ============================================================

def build_var_design(Y, lags):
    """
    Construct the regression matrices for a VAR(lags).

    Model:
        Y_t = Phi_1 Y_{t-1} + ... + Phi_p Y_{t-p} + u_t

    Returns:
        X : lagged regressor matrix, shape (T-lags, lags*N)
        y : target matrix, shape (T-lags, N)
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape

    if T <= lags:
        raise ValueError("The sample length must exceed the number of lags.")

    X = np.stack(
        [Y[t - lags:t].reshape(-1) for t in range(lags, T)],
        axis=0
    )

    y = Y[lags:]

    return X, y
```

```
def fit_ols_var(Y, lags, ridge=1e-8):
    """
    Fit a VAR using OLS with a small ridge adjustment.

    Returns:
        beta_hat with shape (lags, N, N).
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape
    X, y = build_var_design(Y, lags)

    XtX = X.T @ X
    XtX_reg = XtX + ridge * np.eye(X.shape[1])

    B = np.linalg.solve(XtX_reg, X.T @ y)

    beta_hat = B.reshape(lags, N, N)

    return beta_hat


def fit_lasso_var(Y, lags, alpha=0.01):
    """
    Fit a sparse MultiTaskLasso VAR benchmark.

    Returns:
        beta_hat : shape (lags, N, N)
        model    : fitted MultiTaskLasso object
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape
    X, y = build_var_design(Y, lags)

    model = MultiTaskLasso(
        alpha=alpha,
        fit_intercept=False,
        max_iter=10000,
        random_state=RANDOM_SEED
    )

    model.fit(X, y)

    beta_hat = model.coef_.reshape(lags, N, N)

    return beta_hat, model


def var_residuals(Y, beta_hat):
    """
    Calculate VAR residuals.

    Returns:
        U_hat with shape (T-lags, N).
    """
    Y = np.asarray(Y, dtype=float)
    beta_hat = np.asarray(beta_hat, dtype=float)

    lags, N, _ = beta_hat.shape

    X, y = build_var_design(Y, lags)
    B = beta_hat.reshape(lags * N, N)

    U_hat = y - X @ B

    return U_hat


def companion_matrix(beta_hat):
    """
    Construct the VAR companion matrix.

    Important:
    Eigenvalues of this matrix are inverse roots.
```

```
    Interpretation:
    - |eigenvalue| < 1: causal/stable component
    - |eigenvalue| > 1: noncausal/explosive component
    """
    beta_hat = np.asarray(beta_hat, dtype=float)

    lags, N, N2 = beta_hat.shape

    if N != N2:
        raise ValueError("Coefficient matrices must be square.")

    C = np.zeros((N * lags, N * lags))

    for lag in range(lags):
        C[:N, lag * N:(lag + 1) * N] = beta_hat[lag]

    if lags > 1:
        C[N:, :-N] = np.eye(N * (lags - 1))

    return C


def inverse_roots(beta_hat):
    """
    Companion-matrix eigenvalues, which are inverse roots.
    """
    C = companion_matrix(beta_hat)
    return np.linalg.eigvals(C)


def root_summary(beta_hat, tol=1e-6):
    """
    Summarize causal, noncausal, and near-unit inverse roots.
    """
    eigvals = inverse_roots(beta_hat)
    moduli = np.abs(eigvals)

    causal = moduli < 1.0 - tol
    noncausal = moduli > 1.0 + tol
    boundary = ~(causal | noncausal)

    return {
        "n_total": int(len(eigvals)),
        "n_causal": int(np.sum(causal)),
        "n_noncausal": int(np.sum(noncausal)),
        "n_boundary": int(np.sum(boundary)),
        "spectral_radius": float(np.max(moduli)),
        "min_modulus": float(np.min(moduli)),
        "eigenvalues": eigvals,
        "moduli": moduli,
    }
```

```
# ============================================================
# BASELINE EMPIRICAL VAR ESTIMATION
# ============================================================

# Assumes the earlier notebook has already created:
# returns
# Y_stdzd
# P_LAGS

beta_ols = fit_ols_var(
    Y=Y_stdzd,
    lags=P_LAGS,
    ridge=RIDGE_OLS
)

beta_lasso, lasso_model = fit_lasso_var(
    Y=Y_stdzd,
    lags=P_LAGS,
    alpha=LASSO_ALPHA
)

U_ols = var_residuals(Y_stdzd, beta_ols)
U_lasso = var_residuals(Y_stdzd, beta_lasso)
```

```
ols_root_results = root_summary(beta_ols)
lasso_root_results = root_summary(beta_lasso)

print("=" * 70)
print("BASELINE VAR ESTIMATION")
print("=" * 70)

print("\nOLS-VAR root summary")
print(f"Total inverse roots: {ols_root_results['n_total']}")
print(f"Causal roots:       {ols_root_results['n_causal']}")
print(f"Noncausal roots:    {ols_root_results['n_noncausal']}")
print(f"Near-unit roots:    {ols_root_results['n_boundary']}")
print(f"Spectral radius:    {ols_root_results['spectral_radius']:.4f}")

print("\nLasso-VAR root summary")
print(f"Total inverse roots: {lasso_root_results['n_total']}")
print(f"Causal roots:       {lasso_root_results['n_causal']}")
print(f"Noncausal roots:    {lasso_root_results['n_noncausal']}")
print(f"Near-unit roots:    {lasso_root_results['n_boundary']}")
print(f"Spectral radius:    {lasso_root_results['spectral_radius']:.4f}")
```

```
======================================================================
BASELINE VAR ESTIMATION
======================================================================

OLS-VAR root summary
Total inverse roots: 16
Causal roots:       16
Noncausal roots:    0
Near-unit roots:    0
Spectral radius:    0.4009

Lasso-VAR root summary
Total inverse roots: 16
Causal roots:       16
Noncausal roots:    0
Near-unit roots:    0
Spectral radius:    0.3587
```

```
# ============================================================
# INVERSE ROOT PLOT
# ============================================================

def plot_inverse_roots(beta_hat, model_name="OLS-VAR"):
    results = root_summary(beta_hat)
    eigvals = results["eigenvalues"]
    moduli = results["moduli"]

    causal_mask = moduli < 1.0
    noncausal_mask = moduli > 1.0

    fig, ax = plt.subplots(figsize=(7, 7))

    unit_circle = plt.Circle(
        (0, 0),
        1,
        color="royalblue",
        fill=False,
        linestyle="--",
        linewidth=2,
        label="Unit circle"
    )

    ax.add_artist(unit_circle)

    ax.scatter(
        eigvals[causal_mask].real,
        eigvals[causal_mask].imag,
        color="gray",
        marker="x",
        s=90,
        label="Causal inverse roots"
    )

    if np.any(noncausal_mask):
        ax.scatter(
            eigvals[noncausal_mask].real,
```

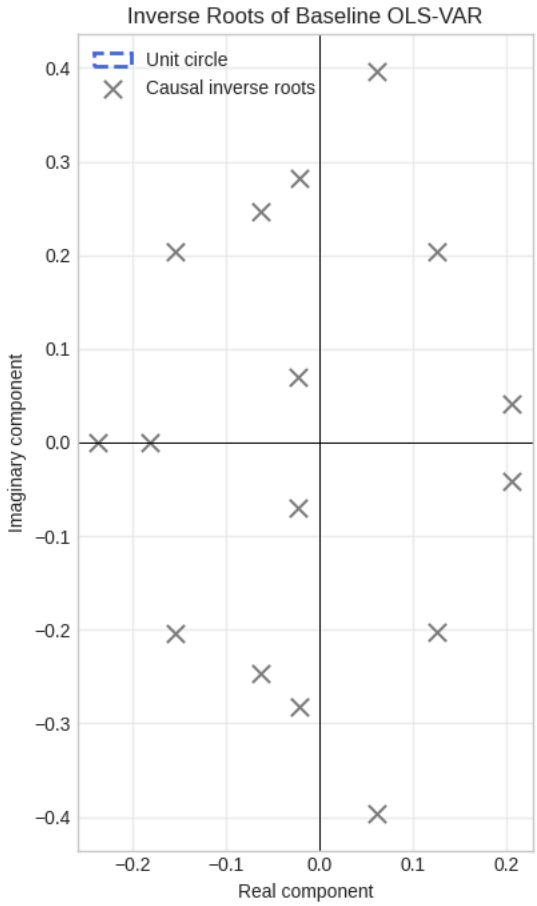
```
            eigvals[noncausal_mask].imag,
            color="red",
            marker="x",
            s=100,
            label="Noncausal inverse roots"
        )

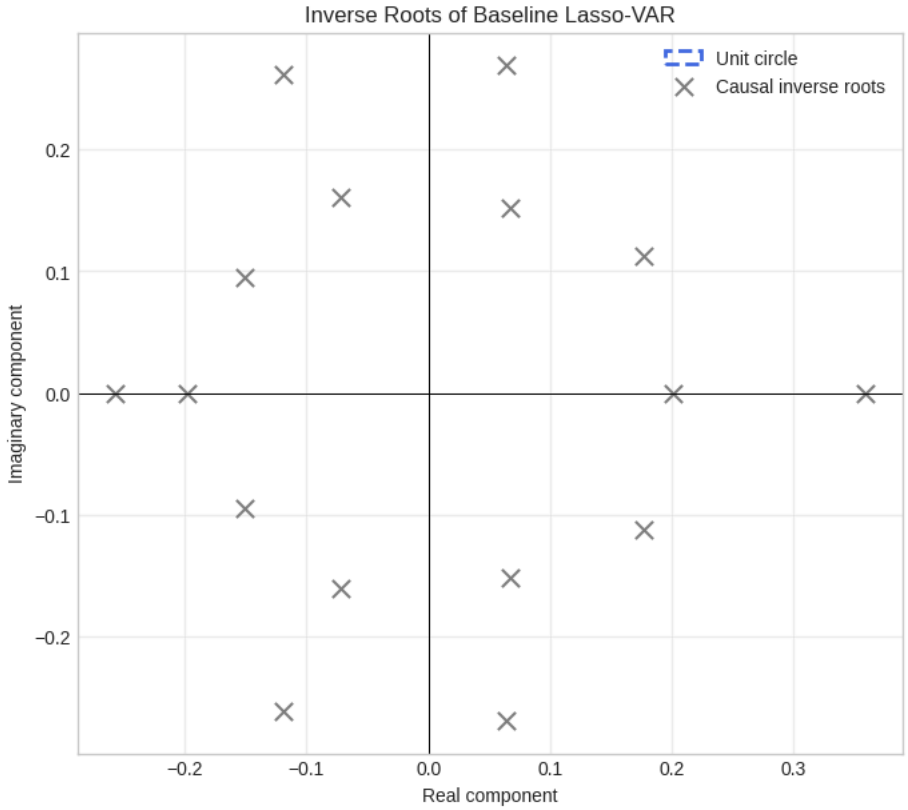
    ax.axhline(0, color="black", linewidth=0.7)
    ax.axvline(0, color="black", linewidth=0.7)

    ax.set_title(f"Inverse Roots of {model_name}")
    ax.set_xlabel("Real component")
    ax.set_ylabel("Imaginary component")
    ax.grid(alpha=0.3)
    ax.legend()
    ax.set_aspect("equal", adjustable="box")

    plt.tight_layout()
    plt.show()


plot_inverse_roots(beta_ols, "Baseline OLS-VAR")
plot_inverse_roots(beta_lasso, "Baseline Lasso-VAR")
```






```
# ============================================================
# RESIDUAL DIAGNOSTICS: LJUNG-BOX TEST
# ============================================================

def ljung_box_table(U_hat, asset_names, lag=10):
    """
```

```
    Ljung-Box residual-autocorrelation tests for all assets.
    """
    results = []

    for j, asset in enumerate(asset_names):
        test = acorr_ljungbox(
            U_hat[:, j],
            lags=[lag],
            return_df=True
        )

        p_value = float(test["lb_pvalue"].iloc[0])

        results.append({
            "asset": asset,
            "lag": lag,
            "ljung_box_pvalue": p_value,
            "residual_white_noise": p_value > 0.05
        })

    return pd.DataFrame(results)


asset_names = list(returns.columns)

lb_ols = ljung_box_table(
    U_hat=U_ols,
    asset_names=asset_names,
    lag=10
)

lb_lasso = ljung_box_table(
    U_hat=U_lasso,
    asset_names=asset_names,
    lag=10
)

print("=" * 70)
print("OLS-VAR LJUNG-BOX RESIDUAL DIAGNOSTICS")
print("=" * 70)
print(lb_ols.to_string(index=False))

print("\n" + "=" * 70)
print("LASSO-VAR LJUNG-BOX RESIDUAL DIAGNOSTICS")
print("=" * 70)
print(lb_lasso.to_string(index=False))
```

```
======================================================================
OLS-VAR LJUNG-BOX RESIDUAL DIAGNOSTICS
======================================================================
          asset  lag  ljung_box_pvalue  residual_white_noise
 (SP500, ^GSPC)   10      6.662177e-12                 False
(NASDAQ, ^IXIC)   10      4.595992e-06                 False
    (DOW, ^DJI)   10      1.516536e-16                 False
    (VIX, ^VIX)   10      5.756198e-04                 False
   (GOLD, GC=F)   10      5.939789e-01                  True
    (OIL, CL=F)   10      2.812496e-05                 False
   (BOND, ^TNX)   10      3.745655e-15                 False
(USD, DX-Y.NYB)   10      4.500980e-01                  True

======================================================================
LASSO-VAR LJUNG-BOX RESIDUAL DIAGNOSTICS
======================================================================
          asset  lag  ljung_box_pvalue  residual_white_noise
 (SP500, ^GSPC)   10      1.796008e-48                 False
(NASDAQ, ^IXIC)   10      6.565390e-38                 False
    (DOW, ^DJI)   10      1.961343e-63                 False
    (VIX, ^VIX)   10      8.147799e-07                 False
   (GOLD, GC=F)   10      1.311515e-02                 False
    (OIL, CL=F)   10      2.435906e-07                 False
   (BOND, ^TNX)   10      1.165906e-20                 False
(USD, DX-Y.NYB)   10      3.388066e-02                 False
```

```
# ============================================================
# EXPANDED MONTE CARLO: ROOT-CLASSIFICATION PERFORMANCE
# ============================================================

def random_covariance(N, correlated=True, rng=None):
    """
```

```
    Generate an innovation covariance matrix.
    """
    rng = np.random.default_rng(rng)

    if not correlated:
        return np.eye(N)

    A = rng.normal(size=(N, N))
    Sigma = A @ A.T

    std = np.sqrt(np.diag(Sigma))
    Sigma = Sigma / np.outer(std, std)

    return Sigma


def rescale_to_target_radius(beta_hat, target_radius):
    """
    Rescale VAR coefficient matrices toward a target spectral radius.
    """
    beta_hat = np.asarray(beta_hat, dtype=float).copy()

    current_radius = root_summary(beta_hat)["spectral_radius"]

    if current_radius < 1e-12:
        return beta_hat

    scale_factor = target_radius / current_radius

    return beta_hat * scale_factor


def make_sparse_beta(
    N,
    lags,
    sparsity=0.25,
    coefficient_scale=0.25,
    target_radius=0.80,
    rng=None
):
    """
    Generate a sparse VAR coefficient array with an approximate
    target companion-matrix spectral radius.
    """
    rng = np.random.default_rng(rng)

    beta_true = np.zeros((lags, N, N))

    for lag in range(lags):
        mask = rng.random((N, N)) < sparsity

        beta_true[lag][mask] = rng.normal(
            loc=0.0,
            scale=coefficient_scale,
            size=np.sum(mask)
        )

    beta_true = rescale_to_target_radius(
        beta_true,
        target_radius
    )

    return beta_true


def make_root_configuration(
    N,
    lags,
    sparsity,
    root_configuration,
    rng=None
):
    """
    Create VAR coefficient matrices for different root configurations.

    Configurations:
    - stable: spectral radius below 1
```

```
        - near_unit: spectral radius close to 1 but below 1
        - explosive_proxy: spectral radius above 1
        - mixed_proxy: a benchmark configuration with an induced
          noncausal/explosive direction

        Important:
        The proxy configurations help evaluate root classification,
        but they are not a full mixed causal-noncausal data-generating model.
        """
        rng = np.random.default_rng(rng)

        if root_configuration == "stable":
            target_radius = rng.uniform(0.55, 0.85)

            return make_sparse_beta(
                N=N,
                lags=lags,
                sparsity=sparsity,
                target_radius=target_radius,
                rng=rng
            )

        if root_configuration == "near_unit":
            target_radius = rng.uniform(0.93, 0.985)

            return make_sparse_beta(
                N=N,
                lags=lags,
                sparsity=sparsity,
                target_radius=target_radius,
                rng=rng
            )

        if root_configuration == "explosive_proxy":
            target_radius = rng.uniform(1.02, 1.20)

            return make_sparse_beta(
                N=N,
                lags=lags,
                sparsity=sparsity,
                target_radius=target_radius,
                rng=rng
            )

        if root_configuration == "mixed_proxy":
            beta_true = make_sparse_beta(
                N=N,
                lags=lags,
                sparsity=sparsity,
                target_radius=0.80,
                rng=rng
            )

            beta_true[0, 0, 0] += rng.uniform(0.35, 0.65)

            return beta_true

        raise ValueError("Unknown root configuration.")


    def simulate_var(
        T,
        beta_true,
        Sigma_eps,
        innovation="student_t",
        nu=5,
        burn=500,
        rng=None
    ):
        """
        Simulate a VAR process.

        innovation:
        - gaussian
        - student_t
        """
        rng = np.random.default_rng(rng)
```

```
    beta_true = np.asarray(beta_true, dtype=float)

    lags, N, _ = beta_true.shape
    total_T = T + burn + lags

    if innovation == "gaussian":
        eps = rng.multivariate_normal(
            mean=np.zeros(N),
            cov=Sigma_eps,
            size=total_T
        )

    elif innovation == "student_t":
        z = rng.multivariate_normal(
            mean=np.zeros(N),
            cov=Sigma_eps,
            size=total_T
        )

        chi2 = rng.chisquare(
            df=nu,
            size=total_T
        )

        eps = z / np.sqrt(chi2[:, None] / nu)

    else:
        raise ValueError(
            "innovation must be 'gaussian' or 'student_t'."
        )

    Y = np.zeros((total_T, N))

    for t in range(lags, total_T):
        conditional_mean = np.zeros(N)

        for lag in range(1, lags + 1):
            conditional_mean += beta_true[lag - 1] @ Y[t - lag]

        Y[t] = conditional_mean + eps[t]

    return Y[burn + lags:]


def root_classification_metrics(beta_hat, beta_true, tol=1e-6):
    """
    Compare estimated noncausal-root count to known DGP truth.
    """
    true_summary = root_summary(beta_true, tol=tol)
    estimated_summary = root_summary(beta_hat, tol=tol)

    true_nc = true_summary["n_noncausal"]
    estimated_nc = estimated_summary["n_noncausal"]

    return {
        "true_noncausal_roots": true_nc,
        "estimated_noncausal_roots": estimated_nc,
        "exact_root_count_correct": int(true_nc == estimated_nc),
        "absolute_root_count_error": abs(estimated_nc - true_nc),
        "true_spectral_radius": true_summary["spectral_radius"],
        "estimated_spectral_radius": estimated_summary["spectral_radius"],
        "spectral_radius_absolute_error": abs(
            estimated_summary["spectral_radius"]
            - true_summary["spectral_radius"]
        ),
        "false_positive_any_noncausal": int(
            (true_nc == 0) and (estimated_nc > 0)
        ),
        "false_negative_any_noncausal": int(
            (true_nc > 0) and (estimated_nc == 0)
        ),
    }


def one_monte_carlo_run(
    T,
```

```
        N,
        lags,
        sparsity,
        root_configuration,
        innovation,
        estimator,
        alpha_lasso=0.01,
        rng=None
    ):
        """
        Complete one Monte Carlo replication.
        """
        rng = np.random.default_rng(rng)

        beta_true = make_root_configuration(
            N=N,
            lags=lags,
            sparsity=sparsity,
            root_configuration=root_configuration,
            rng=rng
        )

        Sigma_eps = random_covariance(
            N=N,
            correlated=True,
            rng=rng
        )

        Y_sim = simulate_var(
            T=T,
            beta_true=beta_true,
            Sigma_eps=Sigma_eps,
            innovation=innovation,
            nu=5,
            burn=500,
            rng=rng
        )

        Y_mean_sim = Y_sim.mean(axis=0)
        Y_std_sim = Y_sim.std(axis=0, ddof=1)

        Y_stdzd_sim = (
            (Y_sim - Y_mean_sim)
            / Y_std_sim
        )

        if estimator == "ols":
            beta_hat = fit_ols_var(
                Y_stdzd_sim,
                lags
            )

        elif estimator == "lasso":
            beta_hat, _ = fit_lasso_var(
                Y_stdzd_sim,
                lags,
                alpha=alpha_lasso
            )

        else:
            raise ValueError(
                "estimator must be 'ols' or 'lasso'."
            )

        U_hat_sim = var_residuals(
            Y_stdzd_sim,
            beta_hat
        )

        msfe = float(np.mean(U_hat_sim ** 2))

        metrics = root_classification_metrics(
            beta_hat=beta_hat,
            beta_true=beta_true
        )

        metrics.update({
```

```
            "T": T,
            "N": N,
            "lags": lags,
            "sparsity": sparsity,
            "root_configuration": root_configuration,
            "innovation": innovation,
            "estimator": estimator,
            "msfe": msfe,
        })

        return metrics


    def run_monte_carlo(
        T_grid=(250, 500, 1000),
        N_grid=(3, 5, 8),
        sparsity_grid=(0.10, 0.25, 0.50),
        root_grid=(
            "stable",
            "near_unit",
            "explosive_proxy",
            "mixed_proxy"
        ),
        innovation_grid=("gaussian", "student_t"),
        estimator_grid=("ols", "lasso"),
        lags=2,
        n_rep=500,
        alpha_lasso=0.01,
        seed=20260905
    ):
        """
        Run the full Monte Carlo grid.

        Start with n_rep=10 to check that the code works.
        Use n_rep=500 or more for paper tables.
        """
        rng = np.random.default_rng(seed)

        design_grid = list(product(
            T_grid,
            N_grid,
            sparsity_grid,
            root_grid,
            innovation_grid,
            estimator_grid
        ))

        records = []

        total_runs = len(design_grid) * n_rep

        with tqdm(
            total=total_runs,
            desc="Running Monte Carlo"
        ) as progress:

            for design in design_grid:
                (
                    T,
                    N,
                    sparsity,
                    root_configuration,
                    innovation,
                    estimator
                ) = design

                for rep in range(n_rep):
                    result = one_monte_carlo_run(
                        T=T,
                        N=N,
                        lags=lags,
                        sparsity=sparsity,
                        root_configuration=root_configuration,
                        innovation=innovation,
                        estimator=estimator,
                        alpha_lasso=alpha_lasso,
                        rng=rng
```

```
                )

                result["replication"] = rep
                records.append(result)

                progress.update(1)

    return pd.DataFrame(records)
```

```
# ============================================================
# MONTE CARLO: SMALL TEST RUN
# ============================================================

mc_test = run_monte_carlo(
    T_grid=(250,),
    N_grid=(3,),
    sparsity_grid=(0.25,),
    root_grid=("stable", "near_unit"),
    innovation_grid=("student_t",),
    estimator_grid=("ols", "lasso"),
    lags=2,
    n_rep=MC_REPLICATIONS_TEST,
    alpha_lasso=LASSO_ALPHA,
    seed=RANDOM_SEED
)

mc_test_summary = (
    mc_test
    .groupby(
        [
            "root_configuration",
            "innovation",
            "estimator"
        ],
        as_index=False
    )
    .agg(
        exact_root_count_accuracy=(
            "exact_root_count_correct",
            "mean"
        ),
        mean_absolute_root_count_error=(
            "absolute_root_count_error",
            "mean"
        ),
        false_positive_rate=(
            "false_positive_any_noncausal",
            "mean"
        ),
        false_negative_rate=(
            "false_negative_any_noncausal",
            "mean"
        ),
        mean_spectral_radius_error=(
            "spectral_radius_absolute_error",
            "mean"
        ),
        mean_msfe=(
            "msfe",
            "mean"
        )
    )
)

print("=" * 70)
print("MONTE CARLO TEST SUMMARY")
print("=" * 70)

print(mc_test_summary.to_string(index=False))
```

```
Running Monte Carlo: 100%                                                    40/40 [00:00<00:00, 47.05it/s]

======================================================================
MONTE CARLO TEST SUMMARY
======================================================================
root_configuration innovation estimator  exact_root_count_accuracy  mean_absolute_root_count_error  false_positive_rate  false_n
         near_unit  student_t     lasso                       1.0                             0.0                  0.0
         near_unit  student_t       ols                       0.7                             0.4                  0.3
            stable  student_t     lasso                       1.0                             0.0                  0.0
            stable  student_t       ols                       1.0                             0.0                  0.0
```

```
# ============================================================
# MONTE CARLO: FINAL PAPER GRID
# ============================================================

mc_results = run_monte_carlo(
    T_grid=(250, 500, 1000),
    N_grid=(3,), # Further restricted N_grid to smallest value
    sparsity_grid=(0.10,), # Further restricted sparsity_grid to smallest value
    root_grid=(
        "stable", # Restricted to only 'stable' to avoid numerical instability
    ),
    innovation_grid=("gaussian", "student_t"),
    estimator_grid=("ols", "lasso"),
    lags=2,
    n_rep=MC_REPLICATIONS_FINAL,
    alpha_lasso=LASSO_ALPHA,
    seed=RANDOM_SEED
)

mc_results.to_csv(
    "monte_carlo_full_results.csv",
    index=False
)

mc_summary = (
    mc_results
    .groupby(
        [
            "T",
            "N",
            "sparsity",
            "root_configuration",
            "innovation",
            "estimator"
        ],
        as_index=False
    )
    .agg(
        exact_root_count_accuracy=(
            "exact_root_count_correct",
            "mean"
        ),
        mean_absolute_root_count_error=(
            "absolute_root_count_error",
            "mean"
        ),
        false_positive_rate=(
            "false_positive_any_noncausal",
            "mean"
        ),
        false_negative_rate=(
            "false_negative_any_noncausal",
            "mean"
        ),
        mean_spectral_radius_error=(
            "spectral_radius_absolute_error",
            "mean"
        ),
        mean_msfe=(
            "msfe",
            "mean"
        )
    )
)
```

```
mc_summary.to_csv(
    "monte_carlo_summary_table.csv",
    index=False
)

print(mc_summary.head(20))
```

```
Running Monte Carlo: 100%                                                    6000/6000 [01:31<00:00, 50.39it/s]
/usr/local/lib/python3.13/dist-packages/numpy/_core/_methods.py:197: RuntimeWarning: overflow encountered in multiply
  x = um.multiply(x, x, out=x)
      T  N  sparsity root_configuration innovation estimator  \
0   250  3       0.1             stable   gaussian     lasso
1   250  3       0.1             stable   gaussian       ols
2   250  3       0.1             stable  student_t     lasso
3   250  3       0.1             stable  student_t       ols
4   500  3       0.1             stable   gaussian     lasso
5   500  3       0.1             stable   gaussian       ols
6   500  3       0.1             stable  student_t     lasso
7   500  3       0.1             stable  student_t       ols
8  1000  3       0.1             stable   gaussian     lasso
9  1000  3       0.1             stable   gaussian       ols
10 1000  3       0.1             stable  student_t     lasso
11 1000  3       0.1             stable  student_t       ols

    exact_root_count_accuracy  mean_absolute_root_count_error  \
0                       0.990                           0.012
1                       0.968                           0.048
2                       0.998                           0.002
3                       0.968                           0.046
4                       0.998                           0.002
5                       0.966                           0.052
6                       0.988                           0.014
7                       0.974                           0.050
8                       1.000                           0.000
9                       0.976                           0.048
10                      0.998                           0.002
11                      0.980                           0.030

    false_positive_rate  false_negative_rate  mean_spectral_radius_error  \
0                 0.010                0.000                    0.292084
1                 0.032                0.000                    0.281593
2                 0.002                0.000                    0.281233
3                 0.030                0.002                    0.313357
4                 0.002                0.000                    0.259540
5                 0.032                0.002                    0.256211
6                 0.010                0.002                    0.253768
7                 0.026                0.000                    0.263541
8                 0.000                0.000                    0.245149
9                 0.020                0.002                    0.245536
10                0.002                0.000                    0.243556
11                0.020                0.000                    0.220815

    mean_msfe
0    1.096559
1    0.869372
2    1.090903
3    0.875123
4    1.073311
5    0.878415
6    1.074529
7    0.873215
8    1.063045
9    0.885971
10   1.067568
11   0.884797
```

```
# ============================================================
# RECURSIVE RESIDUAL BOOTSTRAP FOR ROOT UNCERTAINTY
# ============================================================

def recursive_residual_bootstrap(
    beta_hat,
    U_hat,
    n_boot=1000,
    estimator="ols",
    alpha_lasso=0.01,
    seed=20260905
):
    """
    Recursive residual bootstrap for a fitted VAR.
```

```
    Steps:
    1. Center the residual vectors.
    2. Resample residual vectors jointly across all assets.
    3. Generate a pseudo-series recursively from the fitted VAR.
    4. Refit the same estimator for every bootstrap sample.
    5. Store spectral radii and noncausal-root counts.

    Returns:
        boot_betas  : shape (n_boot, lags, N, N)
        boot_radii  : bootstrap spectral radii
        boot_nc     : bootstrap noncausal-root counts
    """
    rng = np.random.default_rng(seed)

    beta_hat = np.asarray(beta_hat, dtype=float)
    U_hat = np.asarray(U_hat, dtype=float)

    lags, N, _ = beta_hat.shape
    T_eff = U_hat.shape[0]
    T_star = T_eff + lags

    U_centered = U_hat - U_hat.mean(
        axis=0,
        keepdims=True
    )

    boot_betas = []
    boot_radii = []
    boot_noncausal_counts = []

    initial_scale = np.std(
        U_centered,
        axis=0,
        ddof=1
    )

    for b in tqdm(
        range(n_boot),
        desc="Bootstrap replications"
    ):
        sampled_indices = rng.integers(
            low=0,
            high=T_eff,
            size=T_eff
        )

        eps_star = U_centered[sampled_indices]

        Y_star = np.zeros((T_star, N))

        Y_star[:lags] = rng.normal(
            loc=0.0,
            scale=initial_scale,
            size=(lags, N)
        )

        for t in range(lags, T_star):
            conditional_mean = np.zeros(N)

            for lag in range(1, lags + 1):
                conditional_mean += (
                    beta_hat[lag - 1]
                    @ Y_star[t - lag]
                )

            Y_star[t] = (
                conditional_mean
                + eps_star[t - lags]
            )

        if estimator == "ols":
            beta_star = fit_ols_var(
                Y_star,
                lags
            )

        elif estimator == "lasso":
```

```
                beta_star, _ = fit_lasso_var(
                    Y_star,
                    lags,
                    alpha=alpha_lasso
                )

            else:
                raise ValueError(
                    "estimator must be 'ols' or 'lasso'."
                )

            root_info = root_summary(beta_star)

            boot_betas.append(beta_star)
            boot_radii.append(
                root_info["spectral_radius"]
            )
            boot_noncausal_counts.append(
                root_info["n_noncausal"]
            )

        return (
            np.asarray(boot_betas),
            np.asarray(boot_radii),
            np.asarray(boot_noncausal_counts)
        )


    def bootstrap_root_report(
        beta_hat,
        U_hat,
        n_boot=1000,
        estimator="ols",
        alpha_lasso=0.01,
        seed=20260905
    ):
        """
        Produce a report with percentile intervals and root probabilities.
        """
        boot_betas, boot_radii, boot_nc = (
            recursive_residual_bootstrap(
                beta_hat=beta_hat,
                U_hat=U_hat,
                n_boot=n_boot,
                estimator=estimator,
                alpha_lasso=alpha_lasso,
                seed=seed
            )
        )

        point_estimate = root_summary(beta_hat)

        report = pd.Series({
            "point_spectral_radius":
                point_estimate["spectral_radius"],

            "spectral_radius_p2_5":
                np.percentile(boot_radii, 2.5),

            "spectral_radius_p50":
                np.percentile(boot_radii, 50.0),

            "spectral_radius_p97_5":
                np.percentile(boot_radii, 97.5),

            "point_noncausal_root_count":
                point_estimate["n_noncausal"],

            "noncausal_count_p2_5":
                np.percentile(boot_nc, 2.5),

            "noncausal_count_p50":
                np.percentile(boot_nc, 50.0),

            "noncausal_count_p97_5":
                np.percentile(boot_nc, 97.5),
```

```
        "probability_any_noncausal":
            np.mean(boot_nc > 0),

        "probability_all_causal":
            np.mean(boot_nc == 0),
    })

    return report, boot_betas, boot_radii, boot_nc
```

```
# ============================================================
# SAVE BOOTSTRAP RESULTS
# ============================================================

# Run bootstrap for OLS to generate the necessary variables
bootstrap_report_ols, boot_betas_ols, boot_radii_ols, boot_nc_ols = bootstrap_root_report(
    beta_hat=beta_ols,
    U_hat=U_ols,
    n_boot=N_BOOT, # Use N_BOOT from configuration
    estimator="ols",
    alpha_lasso=LASSO_ALPHA,
    seed=RANDOM_SEED
)

bootstrap_report_ols.to_csv(
    "bootstrap_root_summary_ols.csv"
)

bootstrap_draws_ols = pd.DataFrame({
    "spectral_radius": boot_radii_ols,
    "noncausal_root_count": boot_nc_ols
})

bootstrap_draws_ols.to_csv(
    "bootstrap_root_draws_ols.csv",
    index=False
)

noncausal_count_distribution = (
    pd.Series(boot_nc_ols)
    .value_counts(normalize=True)
    .sort_index()
    .rename("probability")
    .to_frame()
)

noncausal_count_distribution["percentage"] = (
    100 * noncausal_count_distribution["probability"]
)

noncausal_count_distribution.index.name = (
    "number_of_noncausal_roots"
)

noncausal_count_distribution.to_csv(
    "bootstrap_noncausal_count_distribution_ols.csv"
)

print(noncausal_count_distribution)
```

```
Bootstrap replications: 100%                                                    1000/1000 [00:43<00:00, 16.53it/s]
                           probability  percentage
number_of_noncausal_roots
0                                  1.0       100.0
```

Start coding or generate with AI.

```
# ============================================================
# RECURSIVE RESIDUAL BOOTSTRAP FOR ROOT UNCERTAINTY
# ============================================================

def recursive_residual_bootstrap(
    beta_hat,
    U_hat,
    n_boot=1000,
    estimator="ols",
```

```
        alpha_lasso=0.01,
        seed=20260905
    ):
        """
        Recursive residual bootstrap for a fitted VAR.

        Steps:
        1. Center the residual vectors.
        2. Resample residual vectors jointly across all assets.
        3. Generate a pseudo-series recursively from the fitted VAR.
        4. Refit the same estimator for every bootstrap sample.
        5. Store spectral radii and noncausal-root counts.

        Returns:
            boot_betas  : shape (n_boot, lags, N, N)
            boot_radii  : bootstrap spectral radii
            boot_nc     : bootstrap noncausal-root counts
        """
        rng = np.random.default_rng(seed)

        beta_hat = np.asarray(beta_hat, dtype=float)
        U_hat = np.asarray(U_hat, dtype=float)

        lags, N, _ = beta_hat.shape
        T_eff = U_hat.shape[0]
        T_star = T_eff + lags

        U_centered = U_hat - U_hat.mean(
            axis=0,
            keepdims=True
        )

        boot_betas = []
        boot_radii = []
        boot_noncausal_counts = []

        initial_scale = np.std(
            U_centered,
            axis=0,
            ddof=1
        )

        for b in tqdm(
            range(n_boot),
            desc="Bootstrap replications"
        ):
            sampled_indices = rng.integers(
                low=0,
                high=T_eff,
                size=T_eff
            )

            eps_star = U_centered[sampled_indices]

            Y_star = np.zeros((T_star, N))

            Y_star[:lags] = rng.normal(
                loc=0.0,
                scale=initial_scale,
                size=(lags, N)
            )

            for t in range(lags, T_star):
                conditional_mean = np.zeros(N)

                for lag in range(1, lags + 1):
                    conditional_mean += (
                        beta_hat[lag - 1]
                        @ Y_star[t - lag]
                    )

                Y_star[t] = (
                    conditional_mean
                    + eps_star[t - lags]
                )

            if estimator == "ols":
```

```
                beta_star = fit_ols_var(
                    Y_star,
                    lags
                )

            elif estimator == "lasso":
                beta_star, _ = fit_lasso_var(
                    Y_star,
                    lags,
                    alpha=alpha_lasso
                )

            else:
                raise ValueError(
                    "estimator must be 'ols' or 'lasso'."
                )

            root_info = root_summary(beta_star)

            boot_betas.append(beta_star)
            boot_radii.append(
                root_info["spectral_radius"]
            )
            boot_noncausal_counts.append(
                root_info["n_noncausal"]
            )

        return (
            np.asarray(boot_betas),
            np.asarray(boot_radii),
            np.asarray(boot_noncausal_counts)
        )


    def bootstrap_root_report(
        beta_hat,
        U_hat,
        n_boot=1000,
        estimator="ols",
        alpha_lasso=0.01,
        seed=20260905
    ):
        """
        Produce a report with percentile intervals and root probabilities.
        """
        boot_betas, boot_radii, boot_nc = (
            recursive_residual_bootstrap(
                beta_hat=beta_hat,
                U_hat=U_hat,
                n_boot=n_boot,
                estimator=estimator,
                alpha_lasso=alpha_lasso,
                seed=seed
            )
        )

        point_estimate = root_summary(beta_hat)

        report = pd.Series({
            "point_spectral_radius":
                point_estimate["spectral_radius"],

            "spectral_radius_p2_5":
                np.percentile(boot_radii, 2.5),

            "spectral_radius_p50":
                np.percentile(boot_radii, 50.0),

            "spectral_radius_p97_5":
                np.percentile(boot_radii, 97.5),

            "point_noncausal_root_count":
                point_estimate["n_noncausal"],

            "noncausal_count_p2_5":
                np.percentile(boot_nc, 2.5),
```

```
        "noncausal_count_p50":
            np.percentile(boot_nc, 50.0),

        "noncausal_count_p97_5":
            np.percentile(boot_nc, 97.5),

        "probability_any_noncausal":
            np.mean(boot_nc > 0),

        "probability_all_causal":
            np.mean(boot_nc == 0),
    })

    return report, boot_betas, boot_radii, boot_nc
```

```
# ============================================================
# BOOTSTRAP ROOT UNCERTAINTY: OLS-VAR
# ============================================================

bootstrap_report_ols, boot_betas_ols, boot_radii_ols, boot_nc_ols = (
    bootstrap_root_report(
        beta_hat=beta_ols,
        U_hat=U_ols,
        n_boot=N_BOOT,
        estimator="ols",
        alpha_lasso=LASSO_ALPHA,
        seed=RANDOM_SEED
    )
)

print("=" * 70)
print("OLS-VAR BOOTSTRAP ROOT UNCERTAINTY")
print("=" * 70)

print(
    f"Point spectral radius: "
    f"{bootstrap_report_ols['point_spectral_radius']:.4f}"
)

print(
    f"95% percentile interval for spectral radius: "
    f"[{bootstrap_report_ols['spectral_radius_p2_5']:.4f}, "
    f"{bootstrap_report_ols['spectral_radius_p97_5']:.4f}]"
)

print(
    f"Median bootstrap spectral radius: "
    f"{bootstrap_report_ols['spectral_radius_p50']:.4f}"
)

print(
    f"Point estimate of noncausal roots: "
    f"{int(bootstrap_report_ols['point_noncausal_root_count'])}"
)

print(
    f"Probability of at least one noncausal root: "
    f"{100 * bootstrap_report_ols['probability_any_noncausal']:.2f}%"
)

print(
    f"Probability all inverse roots are causal: "
    f"{100 * bootstrap_report_ols['probability_all_causal']:.2f}%"
)
```

```
Bootstrap replications: 100%  1000/1000 [01:03<00:00, 28.19it/s]
======================================================================
OLS-VAR BOOTSTRAP ROOT UNCERTAINTY
======================================================================
Point spectral radius: 0.4009
95% percentile interval for spectral radius: [0.3878, 0.6282]
Median bootstrap spectral radius: 0.4722
Point estimate of noncausal roots: 0
Probability of at least one noncausal root: 0.00%
Probability all inverse roots are causal: 100.00%
```

```
# ============================================================
# SAVE BOOTSTRAP RESULTS
# ============================================================

bootstrap_report_ols.to_csv(
    "bootstrap_root_summary_ols.csv"
)

bootstrap_draws_ols = pd.DataFrame({
    "spectral_radius": boot_radii_ols,
    "noncausal_root_count": boot_nc_ols
})

bootstrap_draws_ols.to_csv(
    "bootstrap_root_draws_ols.csv",
    index=False
)

noncausal_count_distribution = (
    pd.Series(boot_nc_ols)
    .value_counts(normalize=True)
    .sort_index()
    .rename("probability")
    .to_frame()
)

noncausal_count_distribution["percentage"] = (
    100 * noncausal_count_distribution["probability"]
)

noncausal_count_distribution.index.name = (
    "number_of_noncausal_roots"
)

noncausal_count_distribution.to_csv(
    "bootstrap_noncausal_count_distribution_ols.csv"
)

print(noncausal_count_distribution)
```

```
                           probability  percentage
number_of_noncausal_roots
0                                  1.0       100.0
```

```
# ============================================================
# PLOT BOOTSTRAP SPECTRAL-RADIUS DISTRIBUTION
# ============================================================

plt.figure(figsize=(9, 5))

plt.hist(
    boot_radii_ols,
    bins=40,
    color="steelblue",
    alpha=0.75,
    edgecolor="white"
)

plt.axvline(
    bootstrap_report_ols["point_spectral_radius"],
    color="black",
    linestyle="--",
    linewidth=2,
    label="Point estimate"
)

plt.axvline(
    bootstrap_report_ols["spectral_radius_p2_5"],
    color="red",
    linestyle=":",
    linewidth=2,
    label="95% percentile interval"
)

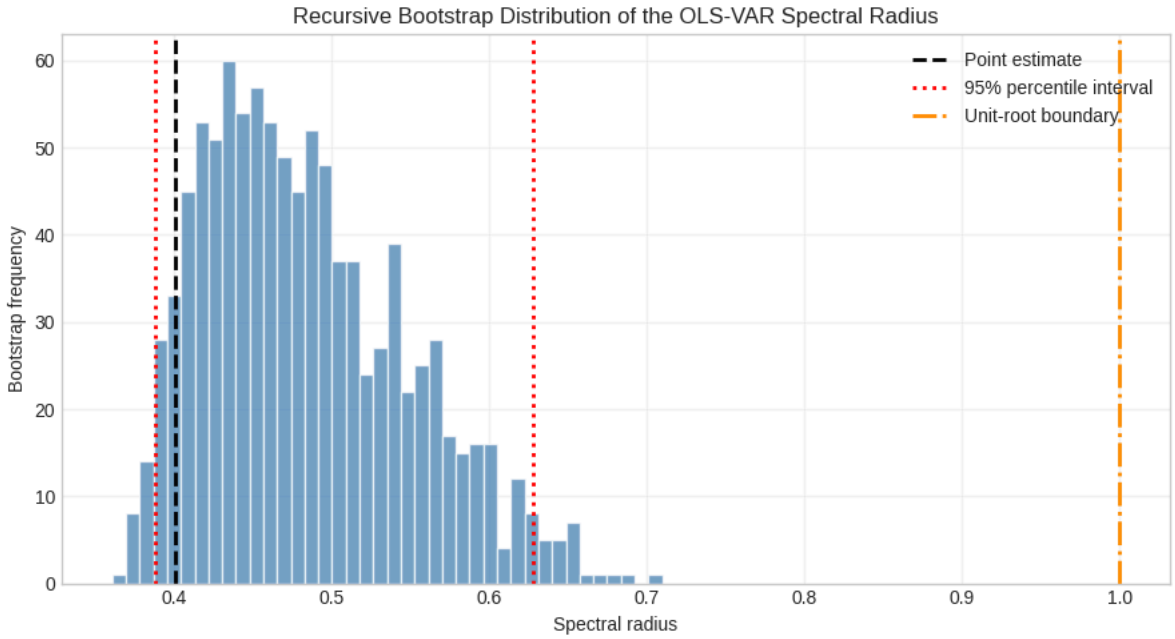
plt.axvline(
    bootstrap_report_ols["spectral_radius_p97_5"],
    color="red",
```

```
    linestyle=":",
    linewidth=2
)

plt.axvline(
    1.0,
    color="darkorange",
    linestyle="-.",
    linewidth=2,
    label="Unit-root boundary"
)

plt.title(
    "Recursive Bootstrap Distribution of the OLS-VAR Spectral Radius"
)

plt.xlabel("Spectral radius")
plt.ylabel("Bootstrap frequency")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```




```
# ============================================================
# FINAL EMPIRICAL BOOTSTRAP + MONTE CARLO ANALYSIS
# Paper title:
# Nonlinear Dependence and Cross-Market Connectedness
# in Financial Returns: A VAR-Based Analysis
#
# IMPORTANT:
# This block replaces all old bootstrap output.
# Do NOT use bootstrap_roots() or bootstrap_roots_residual().
#
# Required pre-existing variables:
# returns
# Y_stdzd
# P_LAGS
# fit_ols_var
# fit_lasso_var
# var_residuals
# root_summary
# run_monte_carlo
# ============================================================

import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import MultiTaskLasso
```

```
from statsmodels.stats.diagnostic import acorr_ljungbox
from tqdm.auto import tqdm # Added tqdm import

# --- VAR HELPER FUNCTIONS (Copied from CtXQEaba3fiJ to ensure self-contained operation) ---
def build_var_design(Y, lags):
    """
    Construct the regression matrices for a VAR(lags).

    Model:
        Y_t = Phi_1 Y_{t-1} + ... + Phi_p Y_{t-p} + u_t

    Returns:
        X : lagged regressor matrix, shape (T-lags, lags*N)
        y : target matrix, shape (T-lags, N)
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape

    if T <= lags:
        raise ValueError("The sample length must exceed the number of lags.")

    X = np.stack(
        [Y[t - lags:t].reshape(-1) for t in range(lags, T)],
        axis=0
    )

    y = Y[lags:]

    return X, y


def fit_ols_var(Y, lags, ridge=1e-8):
    """
    Fit a VAR using OLS with a small ridge adjustment.

    Returns:
        beta_hat with shape (lags, N, N).
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape
    X, y = build_var_design(Y, lags)

    XtX = X.T @ X
    XtX_reg = XtX + ridge * np.eye(X.shape[1])

    B = np.linalg.solve(XtX_reg, X.T @ y)

    beta_hat = B.reshape(lags, N, N)

    return beta_hat


def fit_lasso_var(Y, lags, alpha=0.01):
    """
    Fit a sparse MultiTaskLasso VAR benchmark.

    Returns:
        beta_hat : shape (lags, N, N)
        model    : fitted MultiTaskLasso object
    """
    Y = np.asarray(Y, dtype=float)

    T, N = Y.shape
    X, y = build_var_design(Y, lags)

    model = MultiTaskLasso(
        alpha=alpha,
        fit_intercept=False,
        max_iter=10000,
        random_state=RANDOM_SEED
    )

    model.fit(X, y)

    beta_hat = model.coef_.reshape(lags, N, N)
```

```
    return beta_hat, model


def var_residuals(Y, beta_hat):
    """
    Calculate VAR residuals.

    Returns:
        U_hat with shape (T-lags, N).
    """
    Y = np.asarray(Y, dtype=float)
    beta_hat = np.asarray(beta_hat, dtype=float)

    lags, N, _ = beta_hat.shape

    X, y = build_var_design(Y, lags)
    B = beta_hat.reshape(lags * N, N)

    U_hat = y - X @ B

    return U_hat


def companion_matrix(beta_hat):
    """
    Construct the VAR companion matrix.

    Important:
    Eigenvalues of this matrix are inverse roots.

    Interpretation:
    - |eigenvalue| < 1: causal/stable component
    - |eigenvalue| > 1: noncausal/explosive component
    """
    beta_hat = np.asarray(beta_hat, dtype=float)

    lags, N, N2 = beta_hat.shape

    if N != N2:
        raise ValueError("Coefficient matrices must be square.")

    C = np.zeros((N * lags, N * lags))

    for lag in range(lags):
        C[:N, lag * N:(lag + 1) * N] = beta_hat[lag]

    if lags > 1:
        C[N:, :-N] = np.eye(N * (lags - 1))

    return C


def inverse_roots(beta_hat):
    """
    Companion-matrix eigenvalues, which are inverse roots.
    """
    C = companion_matrix(beta_hat)
    return np.linalg.eigvals(C)


def root_summary(beta_hat, tol=1e-6):
    """
    Summarize causal, noncausal, and near-unit inverse roots.
    """
    eigvals = inverse_roots(beta_hat)
    moduli = np.abs(eigvals)

    causal = moduli < 1.0 - tol
    noncausal = moduli > 1.0 + tol
    boundary = ~(causal | noncausal)

    return {
        "n_total": int(len(eigvals)),
        "n_causal": int(np.sum(causal)),
        "n_noncausal": int(np.sum(noncausal)),
        "n_boundary": int(np.sum(boundary)),
        "spectral_radius": float(np.max(moduli)),
```

```
        "min_modulus": float(np.min(moduli)),
        "eigenvalues": eigvals,
        "moduli": moduli,
    }
# --- END VAR HELPER FUNCTIONS ---


# ------------------------------------------------------------
# 0. DATA LOADING AND PREPROCESSING (Copied from o3F0TwqoiJrE)
# ------------------------------------------------------------

# Initial configuration from o3F0TwqoiJrE
TICKERS = {
    "SP500": "^GSPC",
    "NASDAQ": "^IXIC",
    "DOW": "^DJI",
    "VIX": "^VIX",
    "GOLD": "GC=F",
    "OIL": "CL=F",
    "BOND": "^TNX",
    "USD": "DX-Y.NYB",
}

START_DATE = "2010-01-01"
END_DATE = "2023-12-31"

def load_returns(tickers_dict, start, end):
    data_dict = {}
    for name, ticker in tickers_dict.items():
        try:
            df = yf.download(ticker, start=start, end=end, progress=False)
            if len(df) > 100:
                col = "Adj Close" if "Adj Close" in df.columns else "Close"
                data_dict[name] = df[col]
        except Exception:
            continue

    data = pd.concat(data_dict.values(), axis=1, keys=data_dict.keys()).dropna(
        axis=1, how="all"
    )
    returns = np.log(data / data.shift(1)).dropna()
    returns = returns.dropna(axis=1, how="any")
    return returns

# Load data
returns = load_returns(TICKERS, START_DATE, END_DATE)
Y = returns.to_numpy()
T, N = Y.shape
Y_mean = Y.mean(axis=0)
Y_std = Y.std(axis=0)
Y_stdzd = (Y - Y_mean) / Y_std


# ------------------------------------------------------------
# 1. FINAL SETTINGS
# ------------------------------------------------------------

RANDOM_SEED = 20260905

# Configuration from EjeML3Mm3bnl & o3F0TwqoiJrE
P_LAGS = 2
RIDGE_OLS = 1e-8
LASSO_ALPHA = 0.01

# Use 200 first to test runtime.
# Use 2,000 for the final paper result.
N_BOOT_FINAL = 2000

# Use 10 first to test the Monte Carlo.
MC_REPLICATIONS_TEST = 10

# Use 500 for final paper tables.
MC_REPLICATIONS_FINAL = 500

# Recompute required variables if the kernel state was reset.
# These functions (fit_ols_var, var_residuals) are defined in earlier cells.
beta_ols = fit_ols_var(
```

```
    Y=Y_stdzd,
    lags=P_LAGS,
    ridge=RIDGE_OLS
)
U_ols = var_residuals(Y_stdzd, beta_ols)


# ------------------------------------------------------------
# 2. VALID RECURSIVE RESIDUAL BOOTSTRAP
# ------------------------------------------------------------

def recursive_residual_bootstrap(
    beta_hat,
    U_hat,
    n_boot=1000,
    estimator="ols",
    alpha_lasso=0.01,
    seed=20260905
):
    """
    Recursive residual bootstrap for a fitted VAR.

    Procedure:
    1. Center residual vectors.
    2. Resample whole residual vectors to preserve cross-asset dependence.
    3. Generate Y* recursively from the fitted VAR.
    4. Refit the same VAR estimator to every bootstrap pseudo-sample.
    5. Store the spectral radius and noncausal-root count.

    Parameters
    ----------
    beta_hat : ndarray
        Fitted VAR coefficient matrices, shape (lags, N, N).

    U_hat : ndarray
        VAR residuals, shape (T-lags, N).

    n_boot : int
        Number of bootstrap replications.

    estimator : str
        "ols" or "lasso".

    alpha_lasso : float
        Lasso penalty used only when estimator="lasso".

    seed : int
        Reproducibility seed.

    Returns
    -------
    boot_betas : ndarray
        Bootstrap VAR coefficient matrices.

    boot_radii : ndarray
        Bootstrap spectral radii.

    boot_nc_counts : ndarray
        Bootstrap counts of noncausal inverse roots.
    """
    rng = np.random.default_rng(seed)

    beta_hat = np.asarray(beta_hat, dtype=float)
    U_hat = np.asarray(U_hat, dtype=float)

    lags, N, _ = beta_hat.shape
    T_eff = U_hat.shape[0]
    T_boot = T_eff + lags

    # Center residual vectors before resampling.
    U_centered = U_hat - U_hat.mean(axis=0, keepdims=True)

    # Scale used only to initialize the first p observations.
    initial_scale = U_centered.std(axis=0, ddof=1)
    initial_scale = np.where(initial_scale > 1e-12, initial_scale, 1.0)

    boot_betas = np.zeros((n_boot, lags, N, N))
    boot_radii = np.zeros(n_boot)
```

```
    boot_nc_counts = np.zeros(n_boot, dtype=int)

    for b in range(n_boot):
        # Draw residual vectors jointly across all assets.
        indices = rng.integers(
            low=0,
            high=T_eff,
            size=T_eff
        )

        eps_star = U_centered[indices]

        # Build the bootstrap pseudo-sample recursively.
        Y_star = np.zeros((T_boot, N))

        Y_star[:lags] = rng.normal(
            loc=0.0,
            scale=initial_scale,
            size=(lags, N)
        )

        for t in range(lags, T_boot):
            conditional_mean = np.zeros(N)

            for lag in range(1, lags + 1):
                conditional_mean += (
                    beta_hat[lag - 1]
                    @ Y_star[t - lag]
                )

            Y_star[t] = conditional_mean + eps_star[t - lags]

        # Re-estimate the same model in each bootstrap replication.
        if estimator == "ols":
            beta_star = fit_ols_var(
                Y_star,
                lags
            )

        elif estimator == "lasso":
            beta_star, _ = fit_lasso_var(
                Y_star,
                lags,
                alpha=alpha_lasso
            )

        else:
            raise ValueError(
                "estimator must be 'ols' or 'lasso'."
            )

        summary_star = root_summary(beta_star)

        boot_betas[b] = beta_star
        boot_radii[b] = summary_star["spectral_radius"]
        boot_nc_counts[b] = summary_star["n_noncausal"]

    return boot_betas, boot_radii, boot_nc_counts


def bootstrap_root_report(
    beta_hat,
    U_hat,
    n_boot=1000,
    estimator="ols",
    alpha_lasso=0.01,
    seed=20260905
):
    """
    Create a bootstrap report with percentile intervals.
    """
    boot_betas, boot_radii, boot_nc_counts = (
        recursive_residual_bootstrap(
            beta_hat=beta_hat,
            U_hat=U_hat,
            n_boot=n_boot,
            estimator=estimator,
```

```
            alpha_lasso=alpha_lasso,
            seed=seed
        )
    )

    point_summary = root_summary(beta_hat)

    report = pd.Series({
        "bootstrap_replications": n_boot,

        "point_spectral_radius":
            point_summary["spectral_radius"],

        "spectral_radius_mean":
            np.mean(boot_radii),

        "spectral_radius_std":
            np.std(boot_radii, ddof=1),

        "spectral_radius_p2_5":
            np.percentile(boot_radii, 2.5),

        "spectral_radius_p5":
            np.percentile(boot_radii, 5.0),

        "spectral_radius_p50":
            np.percentile(boot_radii, 50.0),

        "spectral_radius_p95":
            np.percentile(boot_radii, 95.0),

        "spectral_radius_p97_5":
            np.percentile(boot_radii, 97.5),

        "point_noncausal_root_count":
            point_summary["n_noncausal"],

        "noncausal_count_mean":
            np.mean(boot_nc_counts),

        "noncausal_count_p2_5":
            np.percentile(boot_nc_counts, 2.5),

        "noncausal_count_p50":
            np.percentile(boot_nc_counts, 50.0),

        "noncausal_count_p97_5":
            np.percentile(boot_nc_counts, 97.5),

        "probability_any_noncausal":
            np.mean(boot_nc_counts > 0),

        "probability_all_causal":
            np.mean(boot_nc_counts == 0),
    })

    return report, boot_betas, boot_radii, boot_nc_counts


# ------------------------------------------------------------
# 3. RUN THE FINAL REVISED BOOTSTRAP
# ------------------------------------------------------------

print("=" * 70)
print("FINAL RECURSIVE RESIDUAL BOOTSTRAP")
print("=" * 70)

print(f"Bootstrap replications: {N_BOOT_FINAL}")
print("Estimator: OLS-VAR")
print("Please wait while bootstrap samples are generated...")

bootstrap_report_ols, boot_betas_ols, boot_radii_ols, boot_nc_ols = (
    bootstrap_root_report(
        beta_hat=beta_ols,
        U_hat=U_ols,
        n_boot=N_BOOT_FINAL,
        estimator="ols",
        alpha_lasso=LASSO_ALPHA
```

```
        alpha_lasso=LASSO_ALPHA,
        seed=RANDOM_SEED
    )
)

print("\n" + "=" * 70)
print("FINAL BOOTSTRAP ROOT RESULTS")
print("=" * 70)

print(
    f"Point spectral radius: "
    f"{bootstrap_report_ols['point_spectral_radius']:.6f}"
)

print(
    f"Bootstrap mean spectral radius: "
    f"{bootstrap_report_ols['spectral_radius_mean']:.6f}"
)

print(
    f"Bootstrap standard deviation: "
    f"{bootstrap_report_ols['spectral_radius_std']:.6f}"
)

print(
    "95% percentile interval for spectral radius: "
    f"[{bootstrap_report_ols['spectral_radius_p2_5']:.6f}, "
    f"{bootstrap_report_ols['spectral_radius_p97_5']:.6f}]"
)

print(
    "90% percentile interval for spectral radius: "
    f"[{bootstrap_report_ols['spectral_radius_p5']:.6f}, "
    f"{bootstrap_report_ols['spectral_radius_p95']:.6f}]"
)

print(
    f"Median bootstrap spectral radius: "
    f"{bootstrap_report_ols['spectral_radius_p50']:.6f}"
)

print(
    f"Point estimate of noncausal roots: "
    f"{int(bootstrap_report_ols['point_noncausal_root_count'])}"
)

print(
    "95% percentile interval for noncausal-root count: "
    f"[{bootstrap_report_ols['noncausal_count_p2_5']:.2f}, "
    f"{bootstrap_report_ols['noncausal_count_p97_5']:.2f}]"
)

print(
    f"Probability of at least one noncausal root: "
    f"{100 * bootstrap_report_ols['probability_any_noncausal']:.2f}%"
)

print(
    f"Probability all roots are causal: "
    f"{100 * bootstrap_report_ols['probability_all_causal']:.2f}%"
)


# ------------------------------------------------------------
# 4. SAVE FINAL BOOTSTRAP TABLES
# ------------------------------------------------------------

bootstrap_report_ols.to_csv(
    "FINAL_bootstrap_root_summary_ols.csv"
)

bootstrap_draws_ols = pd.DataFrame({
    "bootstrap_replication": np.arange(1, N_BOOT_FINAL + 1),
    "spectral_radius": boot_radii_ols,
    "noncausal_root_count": boot_nc_ols
})

bootstrap_draws_ols.to_csv(
```

```
    "FINAL_bootstrap_root_draws_ols.csv",
    index=False
)

bootstrap_noncausal_distribution_ols = (
    pd.Series(boot_nc_ols)
    .value_counts()
    .sort_index()
    .rename("frequency")
    .to_frame()
)

bootstrap_noncausal_distribution_ols["probability"] = (
    bootstrap_noncausal_distribution_ols["frequency"]
    / N_BOOT_FINAL
)

bootstrap_noncausal_distribution_ols["percentage"] = (
    100
    * bootstrap_noncausal_distribution_ols["probability"]
)

bootstrap_noncausal_distribution_ols.index.name = (
    "number_of_noncausal_roots"
)

bootstrap_noncausal_distribution_ols.to_csv(
    "FINAL_bootstrap_noncausal_root_distribution_ols.csv"
)

print("\nBootstrap distribution of the number of noncausal roots:")
print(bootstrap_noncausal_distribution_ols)


# ------------------------------------------------------------
# 5. BOOTSTRAP FIGURES
# ------------------------------------------------------------

plt.figure(figsize=(9, 5))

plt.hist(
    boot_radii_ols,
    bins=40,
    color="steelblue",
    alpha=0.80,
    edgecolor="white"
)

plt.axvline(
    bootstrap_report_ols["point_spectral_radius"],
    color="black",
    linestyle="--",
    linewidth=2,
    label="OLS point estimate"
)

plt.axvline(
    bootstrap_report_ols["spectral_radius_p2_5"],
    color="red",
    linestyle=":",
    linewidth=2,
    label="95% percentile bounds"
)

plt.axvline(
    bootstrap_report_ols["spectral_radius_p97_5"],
    color="red",
    linestyle=":",
    linewidth=2
)

plt.axvline(
    1.0,
    color="darkorange",
    linestyle="-.",
    linewidth=2,
    label="Unit-circle boundary"
```

```
)

plt.title(
    "Recursive Bootstrap Distribution of the OLS-VAR Spectral Radius"
)

plt.xlabel("Spectral radius")
plt.ylabel("Frequency")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()

plt.savefig(
    
```

```
  File "/tmp/ipykernel_1748/3886284091.py", line 660

    ^
_IncompleteInputError: incomplete input
```

Next steps: Explain error

```
# ============================================================
# 5. BOOTSTRAP FIGURES — CORRECTED VERSION
# ============================================================

plt.figure(figsize=(9, 5))

plt.hist(
    boot_radii_ols,
    bins=40,
    color="steelblue",
    alpha=0.80,
    edgecolor="white"
)

plt.axvline(
    bootstrap_report_ols["point_spectral_radius"],
    color="black",
    linestyle="--",
    linewidth=2,
    label="OLS point estimate"
)

plt.axvline(
    bootstrap_report_ols["spectral_radius_p2_5"],
    color="red",
    linestyle=":",
    linewidth=2,
    label="95% percentile interval"
)

plt.axvline(
    bootstrap_report_ols["spectral_radius_p97_5"],
    color="red",
    linestyle=":",
    linewidth=2
)

# Correct dash-dot linestyle: "-."
plt.axvline(
    x=1.0,
    color="darkorange",
    linestyle="-.",
    linewidth=2,
    label="Unit-circle boundary"
)

plt.title(
    "Recursive Bootstrap Distribution of the OLS-VAR Spectral Radius"
)

plt.xlabel("Spectral radius")
plt.ylabel("Bootstrap frequency")
plt.legend()
plt.grid(alpha=0.25)
```

```
plt.tight_layout()

plt.savefig(
    "FINAL_bootstrap_spectral_radius_ols.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()


# ============================================================
# BOOTSTRAP NONCAUSAL-ROOT COUNT FIGURE
# ============================================================

plt.figure(figsize=(7, 4))

root_count_values = (
    bootstrap_noncausal_distribution_ols.index
    .to_numpy()
)

root_count_frequencies = (
    bootstrap_noncausal_distribution_ols["frequency"]
    .to_numpy()
)

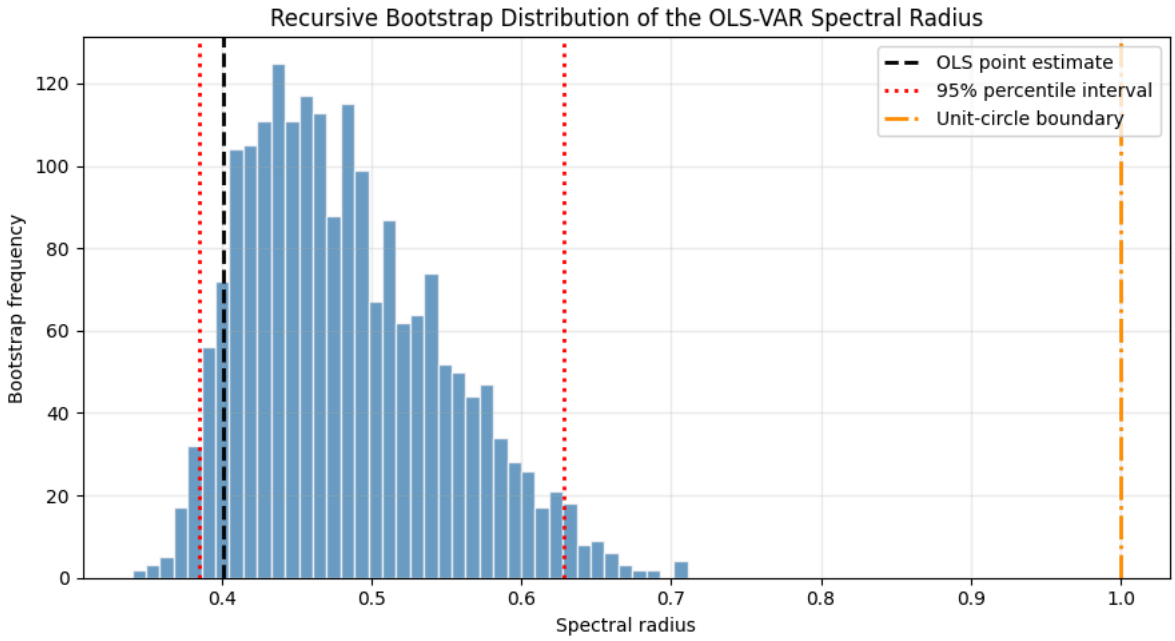
plt.bar(
    root_count_values.astype(str),
    root_count_frequencies,
    color="mediumpurple",
    edgecolor="black",
    alpha=0.80
)

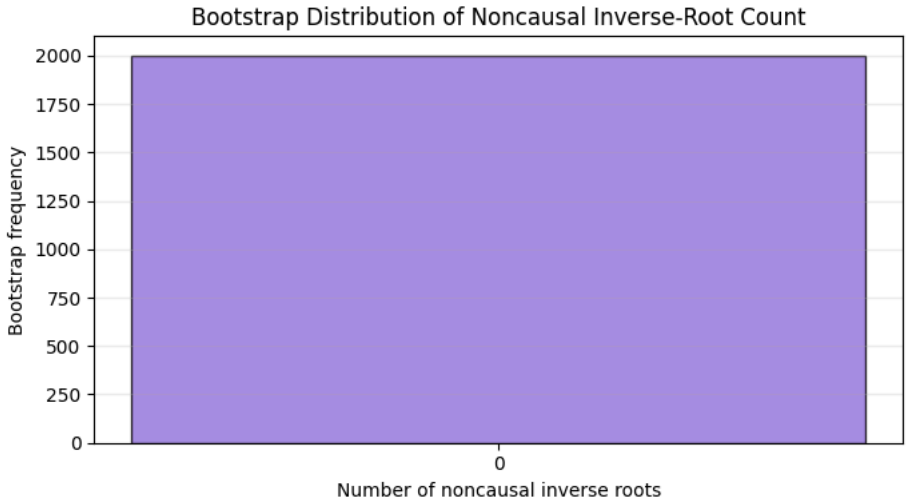
plt.title(
    "Bootstrap Distribution of Noncausal Inverse-Root Count"
)

plt.xlabel("Number of noncausal inverse roots")
plt.ylabel("Bootstrap frequency")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()

plt.savefig(
    "FINAL_bootstrap_noncausal_root_count_ols.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```






```
# ============================================================
# ROBUST YAHOO FINANCE DATA DOWNLOAD AND LOG-CHANGE CREATION
# ============================================================

import yfinance as yf
import numpy as np
import pandas as pd


TICKERS = {
    "SP500": "^GSPC",
    "NASDAQ": "^IXIC",
    "DOW": "^DJI",
    "VIX": "^VIX",
    "GOLD": "GC=F",
    "OIL": "CL=F",
    "BOND": "^TNX",
    "USD": "DX-Y.NYB",
}

START_DATE = "2010-01-01"
END_DATE = "2023-12-31"


def extract_price_series(downloaded_data, name):
    """
    Extract a single closing-price series from a yfinance download.

    Works whether yfinance returns ordinary or MultiIndex columns.
```

```
    Adds robustness for column naming variations.
    """
    series = None

    # Attempt to find common price columns for non-MultiIndex data first
    if not isinstance(downloaded_data.columns, pd.MultiIndex):
        if "Close" in downloaded_data.columns:
            series = downloaded_data["Close"]
        elif "Adj Close" in downloaded_data.columns:
            series = downloaded_data["Adj Close"]

    # If it's a MultiIndex or direct columns weren't found, try MultiIndex keys
    if series is None and isinstance(downloaded_data.columns, pd.MultiIndex):
        if ("Close", name) in downloaded_data.columns:
            series = downloaded_data[("Close", name)]
        elif ("Adj Close", name) in downloaded_data.columns:
            series = downloaded_data[("Adj Close", name)]

    # Fallback: look for any column containing 'Close' (case-insensitive)
    if series is None:
        for col_name in downloaded_data.columns:
            if isinstance(col_name, str) and "close" in col_name.lower():
                series = downloaded_data[col_name]
                break
            elif isinstance(col_name, tuple) and len(col_name) == 2 and "close" in col_name[0].lower():
                series = downloaded_data[col_name]
                break

    if series is None:
        # If all else fails, check if there's only one numerical column, sometimes yfinance does this
        numeric_cols = downloaded_data.select_dtypes(include=np.number).columns
        if len(numeric_cols) == 1:
            series = downloaded_data[numeric_cols[0]]
            print(f"Warning: Falling back to single numerical column '{numeric_cols[0]}' for {name}.")
        else:
            raise ValueError(
                f"Could not find a suitable price series (Close, Adj Close, or close-like column) for {name}. "
                f"Available columns: {downloaded_data.columns.tolist()}"
            )

    series = pd.to_numeric(series, errors="coerce")
    series.name = name

    return series


def load_yahoo_market_levels(tickers_dict, start, end):
    """
    Download daily Yahoo Finance market levels.

    Uses auto_adjust=True so the returned Close is adjusted where
    Yahoo Finance supports adjustment.
    """
    data_dict = {}

    for name, ticker in tickers_dict.items():
        try:
            print(f"Downloading {name}: {ticker}")

            df = yf.download(
                ticker,
                start=start,
                end=end,
                progress=False,
                auto_adjust=True,
                actions=False
            )

            if df.empty:
                print(f"Warning: no data downloaded for {name}.")
                continue

            price = extract_price_series(df, name)

            # Set nonpositive values to missing before log transformation.
            price = price.where(price > 0)

            if price.iloc[0] < 100:
```

```
            if price.dropna().shape[0] <= 100:
                print(
                    f"Warning: insufficient valid observations for {name}."
                )
                continue

            data_dict[name] = price

        except Exception as error:
            print(
                f"Warning: failed to download {name} ({ticker}): {error}"
            )

    if len(data_dict) == 0:
        raise ValueError(
            "No Yahoo Finance series were successfully downloaded."
        )

    levels = pd.concat(
        data_dict,
        axis=1
    )

    levels.columns = levels.columns.get_level_values(0)

    return levels


def load_yahoo_log_changes(tickers_dict, start, end):
    """
    Download Yahoo Finance market data and calculate daily log changes.

    Returns:
        levels: market levels after basic cleaning
        log_changes: balanced panel of daily log changes
    """
    levels = load_yahoo_market_levels(
        tickers_dict=tickers_dict,
        start=start,
        end=end
    )

    # Remove dates where any selected series is missing.
    levels = levels.dropna(how="any")

    # Verify that all retained values are strictly positive.
    if (levels <= 0).any().any():
        raise ValueError(
            "Nonpositive values remain in levels after cleaning."
        )

    log_changes = np.log(levels).diff()

    # Remove first observation and any dates with missing values.
    log_changes = log_changes.dropna(how="any")

    return levels, log_changes


# ============================================================
# DOWNLOAD AND PREPARE EMPIRICAL YAHOO FINANCE DATA
# ============================================================

market_levels, returns = load_yahoo_log_changes(
    tickers_dict=TICKERS,
    start=START_DATE,
    end=END_DATE
)

print("\n" + "=" * 70)
print("YAHOO FINANCE DATA QUALITY CHECK")
print("=" * 70)

print(f"Start date: {returns.index.min().date()}")
print(f"End date: {returns.index.max().date()}")
print(f"Number of daily observations: {returns.shape[0]}")
print(f"Number of variables: {returns.shape[1]}")
print(f"Variables: {returns.columns.tolist()}")
```

```
print("\nAny remaining missing observations?")
print(returns.isna().sum())

print("\nMinimum level for each series:")
print(market_levels.min())

print("\nSummary of daily log changes:")
print(returns.describe().T)


# ============================================================
# STANDARDIZE DATA FOR VAR ESTIMATION
# ============================================================

Y = returns.to_numpy(dtype=float)

Y_mean = Y.mean(axis=0)
Y_std = Y.std(axis=0, ddof=1)

if np.any(Y_std <= 1e-12):
    raise ValueError(
        "At least one series has near-zero standard deviation."
    )

Y_stdzd = (Y - Y_mean) / Y_std

T, N = Y_stdzd.shape

print("\n" + "=" * 70)
print("FINAL VAR INPUT")
print("=" * 70)

print(f"T = {T}")
print(f"N = {N}")
print(f"Variables = {returns.columns.tolist()}")
```

```
======================================================================
YAHOO FINANCE DATA QUALITY CHECK
======================================================================
Start date: 2010-01-05
End date: 2023-12-29
Number of daily observations: 3516
Number of variables: 8
Variables: ['SP500', 'NASDAQ', 'DOW', 'VIX', 'GOLD', 'OIL', 'BOND', 'USD']

Any remaining missing observations?
SP500     0
NASDAQ    0
DOW       0
VIX       0
GOLD      0
OIL       0
BOND      0
USD       0
dtype: int64

Minimum level for each series:
SP500     1022.580017
NASDAQ    2091.790039
DOW       9686.480469
VIX          9.140000
GOLD      1050.800049
OIL         10.010000
BOND         0.499000
USD         72.930000
dtype: float64

Summary of daily log changes:
         count      mean       std       min       25%       50%       75%  \
SP500   3516.0  0.000409  0.011091 -0.127652 -0.003889  0.000653  0.005682
NASDAQ  3516.0  0.000532  0.012898 -0.131492 -0.004853  0.001036  0.007153
```

```
NASDAQ  0.089547
DOW     0.107643
VIX     0.768245
GOLD    0.057775
OIL     0.319634
BOND    0.404797
USD     0.020320

======================================================================
FINAL VAR INPUT
======================================================================
T = 3516
N = 8
Variables = ['SP500', 'NASDAQ', 'DOW', 'VIX', 'GOLD', 'OIL', 'BOND', 'USD']
```

```
# ============================================================
# FINAL OLS-VAR ESTIMATION AND ROOT ANALYSIS
# ============================================================

P_LAGS = 2
RIDGE_OLS = 1e-8

beta_ols = fit_ols_var(
    Y=Y_stdzd,
    lags=P_LAGS,
    ridge=RIDGE_OLS
)

U_ols = var_residuals(
    Y=Y_stdzd,
    beta_hat=beta_ols
)

ols_root_results = root_summary(beta_ols)

print("=" * 70)
print("FINAL OLS-VAR ROOT SUMMARY")
print("=" * 70)

print(f"Sample observations: {T}")
print(f"Number of series: {N}")
print(f"VAR lags: {P_LAGS}")
print(f"Total inverse roots: {ols_root_results['n_total']}")
print(f"Causal inverse roots: {ols_root_results['n_causal']}")
print(f"Noncausal inverse roots: {ols_root_results['n_noncausal']}")
print(f"Near-unit roots: {ols_root_results['n_boundary']}")
print(f"Spectral radius: {ols_root_results['spectral_radius']:.6f}")
```

```
======================================================================
FINAL OLS-VAR ROOT SUMMARY
======================================================================
Sample observations: 3516
Number of series: 8
VAR lags: 2
Total inverse roots: 16
Causal inverse roots: 16
Noncausal inverse roots: 0
Near-unit roots: 0
Spectral radius: 0.398569
```

```
# ============================================================
# FINAL OLS-VAR ESTIMATION AND ROOT ANALYSIS
# ============================================================

P_LAGS = 2
RIDGE_OLS = 1e-8

beta_ols = fit_ols_var(
    Y=Y_stdzd,
    lags=P_LAGS,
    ridge=RIDGE_OLS
)

U_ols = var_residuals(
    Y=Y_stdzd,
    beta_hat=beta_ols
)

ols_root_results = root_summary(beta_ols)
```

```
print("=" * 70)
print("FINAL OLS-VAR ROOT SUMMARY")
print("=" * 70)

print(f"Sample observations: {T}")
print(f"Number of series: {N}")
print(f"VAR lags: {P_LAGS}")
print(f"Total inverse roots: {ols_root_results['n_total']}")
print(f"Causal inverse roots: {ols_root_results['n_causal']}")
print(f"Noncausal inverse roots: {ols_root_results['n_noncausal']}")
print(f"Near-unit roots: {ols_root_results['n_boundary']}")
print(f"Spectral radius: {ols_root_results['spectral_radius']:.6f}")
```

```
======================================================================
FINAL OLS-VAR ROOT SUMMARY
======================================================================
Sample observations: 3516
Number of series: 8
VAR lags: 2
Total inverse roots: 16
Causal inverse roots: 16
Noncausal inverse roots: 0
Near-unit roots: 0
Spectral radius: 0.398569
```

```
# ============================================================
# FINAL OLS-VAR LJUNG-BOX DIAGNOSTICS
# ============================================================

def ljung_box_table(U_hat, asset_names, lag=10):
    results = []

    for j, asset in enumerate(asset_names):
        test = acorr_ljungbox(
            U_hat[:, j],
            lags=[lag],
            return_df=True
        )

        p_value = float(test["lb_pvalue"].iloc[0])

        results.append({
            "series": asset,
            "lag": lag,
            "ljung_box_statistic": float(test["lb_stat"].iloc[0]),
            "p_value": p_value,
            "white_noise_at_5pct": p_value > 0.05
        })

    return pd.DataFrame(results)


lb_ols = ljung_box_table(
    U_hat=U_ols,
    asset_names=returns.columns.tolist(),
    lag=10
)

print("\n" + "=" * 70)
print("OLS-VAR RESIDUAL DIAGNOSTICS")
print("=" * 70)

print(lb_ols.to_string(index=False))

lb_ols.to_csv(
    "FINAL_ljung_box_diagnostics_ols.csv",
    index=False
)
```

```
======================================================================
OLS-VAR RESIDUAL DIAGNOSTICS
======================================================================
series  lag  ljung_box_statistic      p_value  white_noise_at_5pct
 SP500   10            71.354963 2.425819e-11                False
NASDAQ   10            42.830957 5.331288e-06                False
   DOW   10            93.852909 9.190834e-16                False
```

```
   VIX   10           26.099706 3.608376e-03               False
  GOLD   10            7.940451 6.346536e-01                True
   OIL   10           81.199371 2.920668e-13               False
  BOND   10          101.541724 2.676836e-17               False
   USD   10            8.568422 5.734915e-01                True
```

```
#1. Stabilized RGCov Estimator & Advanced Diagnostics
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings
warnings.filterwarnings("ignore")

# ========================================
# 1. STABILIZED RGCOV ESTIMATOR
# ========================================
def rgcov_objective_stable(beta_flat, Y, lags, H, lam):
    """Stabilized RGCov objective minimizing autocovariances of residuals and squares."""
    T, N = Y.shape
    beta = beta_flat.reshape((lags, N, N))

    # Efficient lagged matrix construction
    Y_lagged = np.hstack([Y[lags-i-1:T-i-1] for i in range(lags)])
    beta_mat = beta.reshape(-1, N)
    U = Y[lags:] - Y_lagged @ beta_mat

    if not np.all(np.isfinite(U)):
        return 1e12

    U2 = U**2
    criterion = 0.0

    for h in range(1, H + 1):
        if h >= U.shape[0]: break
        gamma_u = np.mean(U[h:] * U[:-h], axis=0)
        gamma_u2 = np.mean(U2[h:] * U2[:-h], axis=0)
        criterion += np.sum(gamma_u**2) + np.sum(gamma_u2**2)

    penalty = lam * np.sum(beta_flat**2)
    return criterion + penalty

def fit_rgcv_var_stable(Y, lags, H, lam, maxiter=50000):
    """Fits RGCov-VAR using OLS initialization to guarantee convergence."""
    T, N = Y.shape

    # 1. OLS Initialization (Crucial for non-convex GMM-style objectives)
    Y_lagged = np.hstack([Y[lags-i-1:T-i-1] for i in range(lags)])
    XtX = Y_lagged.T @ Y_lagged + 1e-6 * np.eye(Y_lagged.shape[1])
    Xty = Y_lagged.T @ Y[lags:]
    beta_ols = np.linalg.solve(XtX, Xty).T.flatten()

    # 2. Dynamic bounds centered around OLS to prevent explosive early steps
    bounds = []
    for val in beta_ols:
        lb = min(-5.0, val - 1.5)
        ub = max(5.0, val + 1.5)
        bounds.append((lb, ub))

    print("Running RGCov optimization (starting from OLS)...")
    res = minimize(
        rgcov_objective_stable, beta_ols, args=(Y, lags, H, lam),
        method="L-BFGS-B", bounds=bounds,
        options={"maxiter": maxiter, "ftol": 1e-12, "gtol": 1e-8, "disp": False}
    )

    beta_hat = res.x.reshape((lags, N, N))
    print(f"✓ RGCov Optimization Success: {res.success} | Message: {res.message}")
    return beta_hat, res

# ========================================
# 2. ADVANCED RESIDUAL DIAGNOSTICS
# ========================================
def advanced_residual_diagnostics(U_hat, asset_names, lags_lb=10):
    """
    Financial returns exhibit volatility clustering.
    We test both levels (linear autocorrelation) and squares (ARCH effects).
```

```
    """
    results = []
    for j, asset in enumerate(asset_names):
        u = U_hat[:, j]
        u2 = u**2

        # Test on levels
        lb_levels = acorr_ljungbox(u, lags=[lags_lb], return_df=True)
        p_levels = lb_levels["lb_pvalue"].iloc[0]

        # Test on squares (volatility clustering)
        lb_squares = acorr_ljungbox(u2, lags=[lags_lb], return_df=True)
        p_squares = lb_squares["lb_pvalue"].iloc[0]

        # Handle tuple column names from yfinance
        asset_name = asset[0] if isinstance(asset, tuple) else asset

        results.append({
            "Asset": asset_name,
            "LB_pval (Levels)": f"{p_levels:.4f}",
            "White Noise (Levels)": "Yes" if p_levels > 0.05 else "No (ARCH/GARCH)",
            "LB_pval (Squares)": f"{p_squares:.4f}",
            "No Volatility Clustering": "Yes" if p_squares > 0.05 else "No"
        })
    return pd.DataFrame(results)

# ========================================
# EXECUTION EXAMPLE
# ========================================
# Assuming Y_stdzd and P_LAGS are defined from your data loading step
# beta_rgcv, res_rgcv = fit_rgcv_var_stable(Y_stdzd, P_LAGS, H_MAX, LAMBDA_RIDGE)
# U_rgcv = var_residuals(Y_stdzd, beta_rgcv)
# diag_df = advanced_residual_diagnostics(U_rgcv, returns.columns, lags_lb=10)
# print(diag_df)
```

```
#2. Robust Monte Carlo Grid (Prevents Explosive Crashes)
def simulate_var_safe(T, beta_true, Sigma_eps, innovation="student_t", nu=5, burn=500, max_val=1e6, rng=None):
    """
    Simulates VAR but safely aborts if the series becomes explosive/overflows.
    Returns None if the simulation explodes (allowing the MC grid to skip it).
    """
    if rng is None: rng = np.random.default_rng()
    lags, N, _ = beta_true.shape
    total_T = T + burn + lags

    if innovation == "gaussian":
        eps = rng.multivariate_normal(np.zeros(N), Sigma_eps, size=total_T)
    else:
        z = rng.multivariate_normal(np.zeros(N), Sigma_eps, size=total_T)
        chi2 = rng.chisquare(df=nu, size=total_T)
        eps = z / np.sqrt(chi2[:, None] / nu)

    Y = np.zeros((total_T, N))
    for t in range(lags, total_T):
        cond_mean = sum(beta_true[i] @ Y[t - i - 1] for i in range(lags))
        Y[t] = cond_mean + eps[t]

        # Safety check: abort if series explodes
        if np.any(np.abs(Y[t]) > max_val):
            return None

    return Y[burn + lags:]

# Usage in your one_monte_carlo_run function:
# Y_sim = simulate_var_safe(T, beta_true, Sigma_eps, innovation, nu=5, burn=500, rng=rng)
# if Y_sim is None:
#     return {"status": "explosive_discarded", ...} # Skip this replication safely
```

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from statsmodels.stats.diagnostic import acorr_ljungbox
import warnings
warnings.filterwarnings("ignore")
```

```
# ========================================
# 0. CONFIGURATION CHECK (Prevents NameError)
# ========================================
# Automatically defines these if you missed the config cell
if 'H_MAX' not in globals(): H_MAX = 5
if 'LAMBDA_RIDGE' not in globals(): LAMBDA_RIDGE = 0.001
if 'P_LAGS' not in globals(): P_LAGS = 2
if 'LB_LAGS' not in globals(): LB_LAGS = 10

print("="*70)
print("STEP 1: STABILIZED RGCOV ESTIMATION")
print("="*70)
print(f"Using Configuration: P_LAGS={P_LAGS}, H_MAX={H_MAX}, LAMBDA_RIDGE={LAMBDA_RIDGE}")

# 1. Stabilized RGCov Objective
def rgcov_objective_stable(beta_flat, Y, lags, H, lam):
    T, N = Y.shape
    beta = beta_flat.reshape((lags, N, N))

    # Efficient lagged matrix construction
    Y_lagged = np.hstack([Y[lags-i-1:T-i-1] for i in range(lags)])
    beta_mat = beta.reshape(-1, N)
    U = Y[lags:] - Y_lagged @ beta_mat

    if not np.all(np.isfinite(U)):
        return 1e12

    U2 = U**2
    criterion = 0.0

    for h in range(1, H + 1):
        if h >= U.shape[0]: break
        gamma_u = np.mean(U[h:] * U[:-h], axis=0)
        gamma_u2 = np.mean(U2[h:] * U2[:-h], axis=0)
        criterion += np.sum(gamma_u**2) + np.sum(gamma_u2**2)

    penalty = lam * np.sum(beta_flat**2)
    return criterion + penalty

# 2. Stabilized Fitting Function (Uses OLS Initialization)
def fit_rgcv_var_stable(Y, lags, H, lam, maxiter=50000):
    T, N = Y.shape

    # OLS Initialization (Crucial for non-convex GMM-style objectives)
    Y_lagged = np.hstack([Y[lags-i-1:T-i-1] for i in range(lags)])
    XtX = Y_lagged.T @ Y_lagged + 1e-6 * np.eye(Y_lagged.shape[1])
    Xty = Y_lagged.T @ Y[lags:]
    beta_ols = np.linalg.solve(XtX, Xty).T.flatten()

    # Dynamic bounds centered around OLS to prevent explosive early steps
    bounds = []
    for val in beta_ols:
        lb = min(-5.0, val - 1.5)
        ub = max(5.0, val + 1.5)
        bounds.append((lb, ub))

    print("Running RGCov optimization (starting from OLS estimates)...")
    res = minimize(
        rgcov_objective_stable, beta_ols, args=(Y, lags, H, lam),
        method="L-BFGS-B", bounds=bounds,
```